

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Olken, Benjamin et al. "Nonparametric Regression and Measurement Error. " Development Economics: Microeconomic Issues and Policy Models, Fall 2008, Massachusetts Institute of Technology. Lecture: <https://drive.google.com/file/d/19nVUBWNkPzbFXFaNB13Y9bc-tz1xDkzR/view?usp=sharing>

- **Required pages:** Entire lecture
- **Estimated reading time:** 30 minutes

2. Hartmann, K., Krois, J., and Waske, B. *E-Learning Project SOGA: Statistics and Geospatial Data Analysis*. Department of Earth Sciences, Freie Universitaet Berlin, 2018, <https://www.geo.fu-berlin.de/en/v/soga-py/index.html>

- **Required pages:**
 - Factor Analysis: <https://www.geo.fu-berlin.de/en/v/soga-r/Advances-statistics/Multivariate-approaches/Factor-Analysis/index.html>
 - The Exploratory Factor Model (EFM): <https://www.geo.fu-berlin.de/en/v/soga-r/Advances-statistics/Multivariate-approaches/Factor-Analysis/The-Exploratory-Factor-Model/index.html>
 - Explained and Unexplained Variability: <https://www.geo.fu-berlin.de/en/v/soga-r/Advances-statistics/Multivariate-approaches/Factor-Analysis/The-Exploratory-Factor-Model/Explained-and-Unexplained-Variability/index.html>
 - Parameter Estimation: <https://www.geo.fu-berlin.de/en/v/soga-r/Advances-statistics/Multivariate-approaches/Factor-Analysis/The-Exploratory-Factor-Model/Parameter-Estimation/index.html>
 - Factor Rotation: <https://www.geo.fu-berlin.de/en/v/soga-r/Advances-statistics/Multivariate-approaches/Factor-Analysis/The-Exploratory-Factor-Model/Factor-Rotation/index.html>
 - Confusion with Principal Component Analysis: <https://www.geo.fu-berlin.de/en/v/soga-r/Advances-statistics/Multivariate-approaches/Factor-Analysis/The-Exploratory-Factor-Model/Confusion-with-PCA/index.html>
 - A Simple Example of Factor Analysis in R: <https://www.geo.fu-berlin.de/en/v/soga-r/Advances-statistics/Multivariate-approaches/Factor-Analysis/A-Simple-Example-of-Factor-Analysis-in-R/index.html>
- **Estimated reading time:** 2 hours

FINANCIAL ECONOMETRICS

MODULE 7 | LESSON 1

MEASUREMENT ERRORS AND FACTOR ANALYSIS

Reading Time	60 minutes
Prior Knowledge	Linear Algebra, Linear Regression, Principal Component Analysis
Keywords	Measurement Error, Exploratory Factor Analysis, Confirmatory Factor Analysis, Factors/Latent Variables, Common Variance/Communality, Unique Variance, Loadings, Varimax Rotation, Promax Rotation

In previous lessons, we assumed that the data we get are always accurate. In reality, the collected values of the data we get can be different from the actual values of the data. This difference is called measurement error. In this lesson, we will examine the impact of measurement error in a linear regression model. With the knowledge of measurement error, we will be able to discuss the next topic: factor analysis. Factor analysis is an analytical method to understand the behavior of the observed dataset we get. We will also learn how to use factor analysis to model measurement error. Factor analysis is another popular method to reduce the data dimension.

```
In [1]: import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from factor_analyzer import FactorAnalyzer
from factor_analyzer.factor_analyzer import calculate_bartlett_sphericity, calculate_kmo, calculate_msa, calculate_mt, calculate_vmr, calculate_wm, calculate_z
from sklearn import preprocessing

plt.rcParams["figure.figsize"] = (12, 9)
```

```
In [2]: # Download the dataset
data_set = pd.read_csv("fin_data_set.csv")
```

1. Measurement Error

1.1 Overview

What is measurement error? **Measurement error** arises when the measured values of the data are different from the true values of the data. One example is when we measure our weight on a scale, the measured number on the scale can be incorrect. The read can be five pounds more or less than the actual weight. We would take the measurement several times and calculate the average. Then, we use the average as the final weight. The difference between each measurement and the average is the measurement error. This is a situation when the instrument or method of measurement is not precise.

Another situation where measurement error can occur is when a variable is a conceptual idea but difficult to quantify or observe. For example, when we try to evaluate how strong a country's economy is, the first variable we usually come up with is GDP, but GDP only evaluates transacted goods, not the labor situation. You might want to include employment rate and other measures in your analysis. Following that logic, you might collect a number of variables to approximate a country's economy performance. However, there is still a difference between the economy's performance and the combined effect of the selected variables.

Measurement error can be divided into two types of errors: systematic error and random error. **Systematic error** is an error that can be identified. The above examples are all identifiable errors, like inaccurate instruments or imperfect proxies. **Random error** is the error we cannot identify, even after correcting all identifiable errors. It is an error that always exists in measurements and fluctuates. What we can do is take many measurements and use the average to even out the fluctuation.

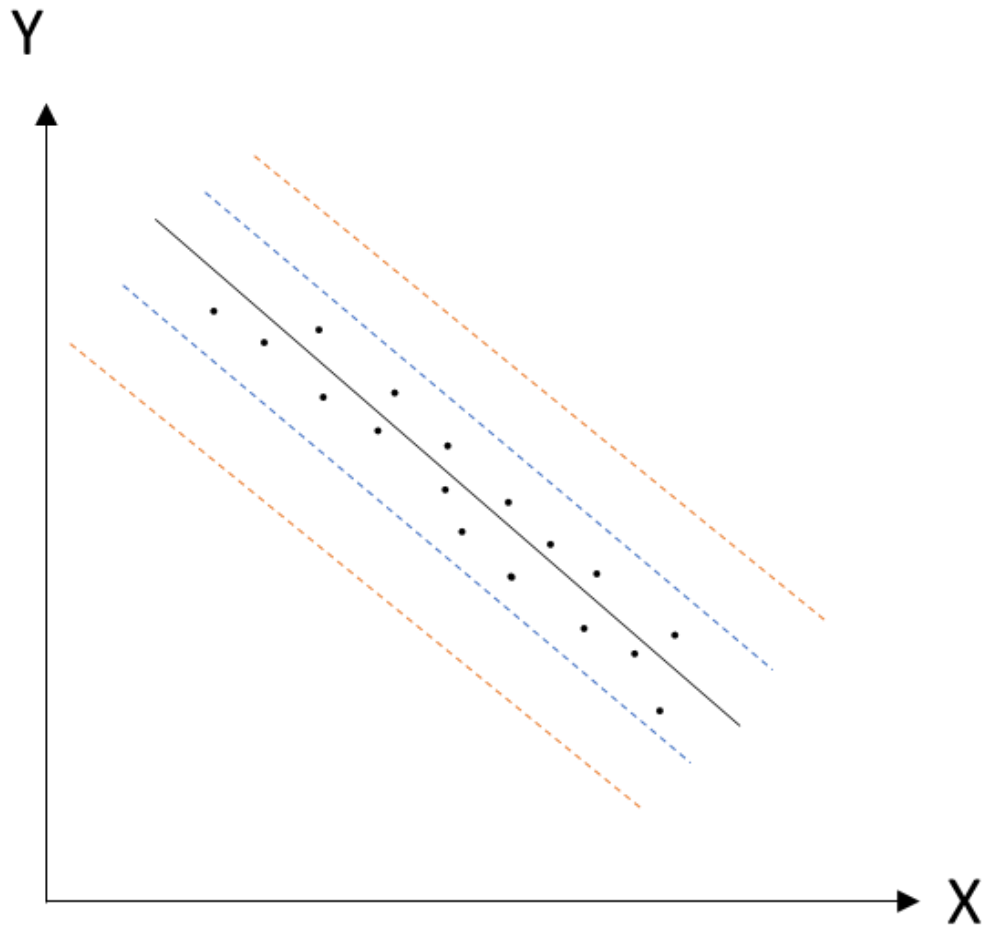
1.2 Impact of Measurement Error in Linear Regression

Now, let's look at the variables with measurement error in a linear regression model. What can happen to coefficient estimates when variables in a linear regression model have measurement errors?

1.2.1 Dependent Variable with Measurement Error

If the dependent variable in a linear regression has measurement error, the coefficient estimates are still unbiased. However, the variances of the coefficient estimates will be larger and the confidence intervals for coefficient estimates will be wider. Hence, the estimates will be less stable. Figure 1 shows the consequence of a dependent variable with measurement error and the confidence intervals. The blue dotted line is the confidence interval without measurement error. The red dotted line represents the confidence interval with measurement error.

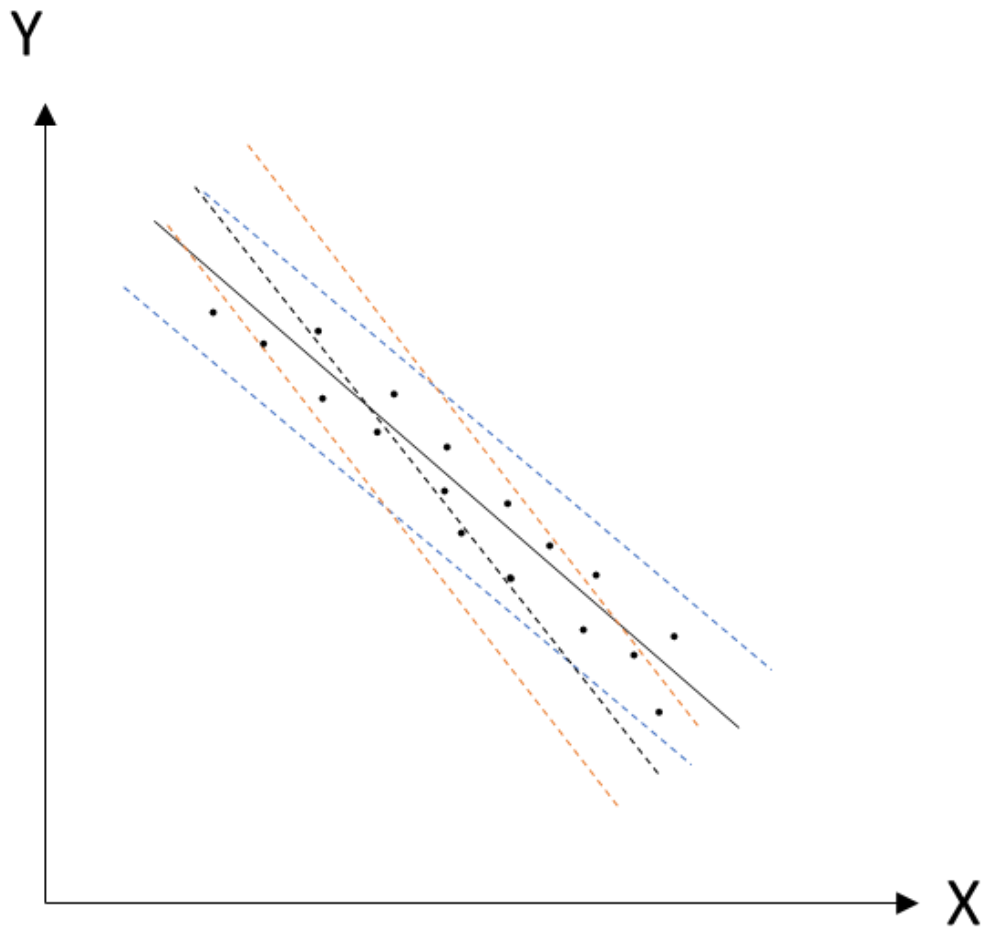
Figure 1: Dependent Variable and Measurement Error in a Linear Regression Model



1.2.2 Independent Variables with Measurement Error

If an independent variable in a linear regression has measurement error, the coefficient estimate of the independent variable will be biased. Figure 2 demonstrates different regression lines for the independent variable with and without measurement error. The black solid line is the independent variable without measurement error. The black dotted line is the independent variable with measurement error. We can see now that the slope for the black solid line and the slope for the black dotted line are different. The slopes represent coefficient estimates for two linear regression models.

Figure 2: Independent Variable and Measurement Error in a Linear Regression Model



1.2.3 What Do You Do with Measurement Errors?

We know part of the reasons measurement error exists is because we are using a number of variables as a proxy for an idea or a concept, as described in the last section. Measurement error is the difference between the proxy value and the true value. From the example in the last section, a country's economy performance is evaluated by using a number of measured variables. To look at the method in a different way, these measured variables are all influenced by a variable that we cannot directly measure. The measurement error is the difference between this unmeasurable variable and the combined measurable variables. If we can find an analytical tool to model this relationship, we can model the measurement error. This analytical tool is **factor analysis**. We are going to talk about factor analysis in the next section.

2. Factor Analysis

2.1 Factors and Latent Variables

Before discussing factor analysis, we would like to introduce a few key terms. In the last section, we talked about using several measurable variables to approximate the unmeasurable variable. To look at this relationship in another way, we can say that the unmeasurable variable also has influence on the several measurable variables. We call the measurable variables or observed variables **indicators**. We call the unmeasurable variable a **factor**. A **factor** is an unobserved variable that can influence the variances of observed variables. Since we cannot observe a factor, we also call it a **latent variable**. Assuming we have three indicators (Y_1 , Y_2 , and Y_3) and one latent variable (F_1), we can illustrate their relationship as the following linear relationship:

$$\begin{aligned} Y_1 &= b_1 F_1 + \epsilon_1 \\ Y_2 &= b_2 F_1 + \epsilon_2 \\ Y_3 &= b_3 F_1 + \epsilon_3 \end{aligned}$$

From the above linear system, we can see F_1 is the common factor for the three indicators. The total variance of an indicator can be split between **common variance** and **unique variance**. The **common variance** is the variance that is explained by the factor. We also call common variance **communality**. The **unique variance** is the indicator's specific variance. In our example above, the common variance for Y_1 is the variance explained by F_1 . The unique variance is the variance explained by ϵ_1 which is the measurement error we discussed in the last section. Hence, this system can let us model both factors shared among indicators and indicators' specific measurement error.

There can be several factors that influence indicators. If there are two factors (F_1 and F_2) influencing Y_1 , Y_2 , and Y_3 , we can write our linear system as follows:

$$\begin{aligned} Y_1 &= b_1 F_1 + c_1 F_2 + \epsilon_1 \\ Y_2 &= b_2 F_1 + c_2 F_2 + \epsilon_2 \\ Y_3 &= b_3 F_1 + c_3 F_2 + \epsilon_3 \end{aligned}$$

The coefficients b_1 , b_2 , b_3 , c_1 , c_2 , and c_3 are called **factor loadings** or **loadings**. Loadings tell us which indicators are dominant for a factor.

2.2 Basics of Factor Analysis (Exploratory Factor Analysis)

There are two types of factor analysis: exploratory factor analysis and confirmatory factor analysis. Exploratory factor analysis is used to find possible factors underlying a group of indicators without assuming any structure of the factors. Confirmatory factor analysis is applied to test hypothetical structures of factors on indicators. In this class, we will focus on exploratory factor analysis (EFA).

Why do we need to use factor analysis? As we described above, we can now model measurement error. The main reason most researchers use factor analysis is to find and use factors to understand a group of variables. A researcher may have a large number of variables to investigate; factor analysis will help find a few key factors to explain the whole group of variables. The number of factors will be less than the number of the initial variables.

Therefore, factor analysis is frequently used to reduce the number of variables. Factor analysis is also a technique to **reduce the dimension** of a dataset.

The general form of a factor analysis model looks like the following:

$$\begin{aligned} Y_1 &= b_{11} F_1 + b_{12} F_2 + \dots + b_{1m} F_m + \epsilon_1 \\ Y_2 &= b_{21} F_1 + b_{22} F_2 + \dots + b_{2m} F_m + \epsilon_2 \\ Y_3 &= b_{31} F_1 + b_{32} F_2 + \dots + b_{3m} F_m + \epsilon_3 \\ &\vdots \\ Y_n &= b_{n1} F_1 + b_{n2} F_2 + \dots + b_{nm} F_m + \epsilon_n \end{aligned}$$

In the above model, there are n indicators and m factors. (b_{11} , b_{21} , b_{31} , ... b_{n1}) are the factor loadings for factor 1. The goal of factor analysis is to determine the optimal number of factors to explain the variables, estimate the loadings, and provide possible ideas to form groups of variables using the information from factors.

2.3 An Example of Factor Analysis

The best way to understand how factor analysis works is to look at an example. We will continue to use our *KO* excess return model example. In the model, we have the following eight independent variables:

```
Dow 30 excess return
Pepsi stock excess return
KO revenue growth
Google stock excess return
Bank of America stock excess return
Walmart stock excess return
General Electric stock excess return
Starbucks stock excess return
```

By using factor analysis, we would like to find fewer than eight factors, and the factors can explain the most variance from the eight independent variables.

First we need to standardise our data.

```
In [14]: # standardise data
data = data_set[
    [
        "Dow_Q_EX_R",
        "Pepsi_Q_EX_R",
        "REV_Q_R",
        "GOOG_Q_EX_R",
        "BAC_Q_EX_R",
        "WMT_Q_EX_R",
        "GE_Q_EX_R",
        "SBUX_Q_EX_R",
    ]
]
```

```
data_normal = preprocessing.StandardScaler().fit_transform(data)
df = pd.DataFrame(data_normal, columns=[data.columns])
```

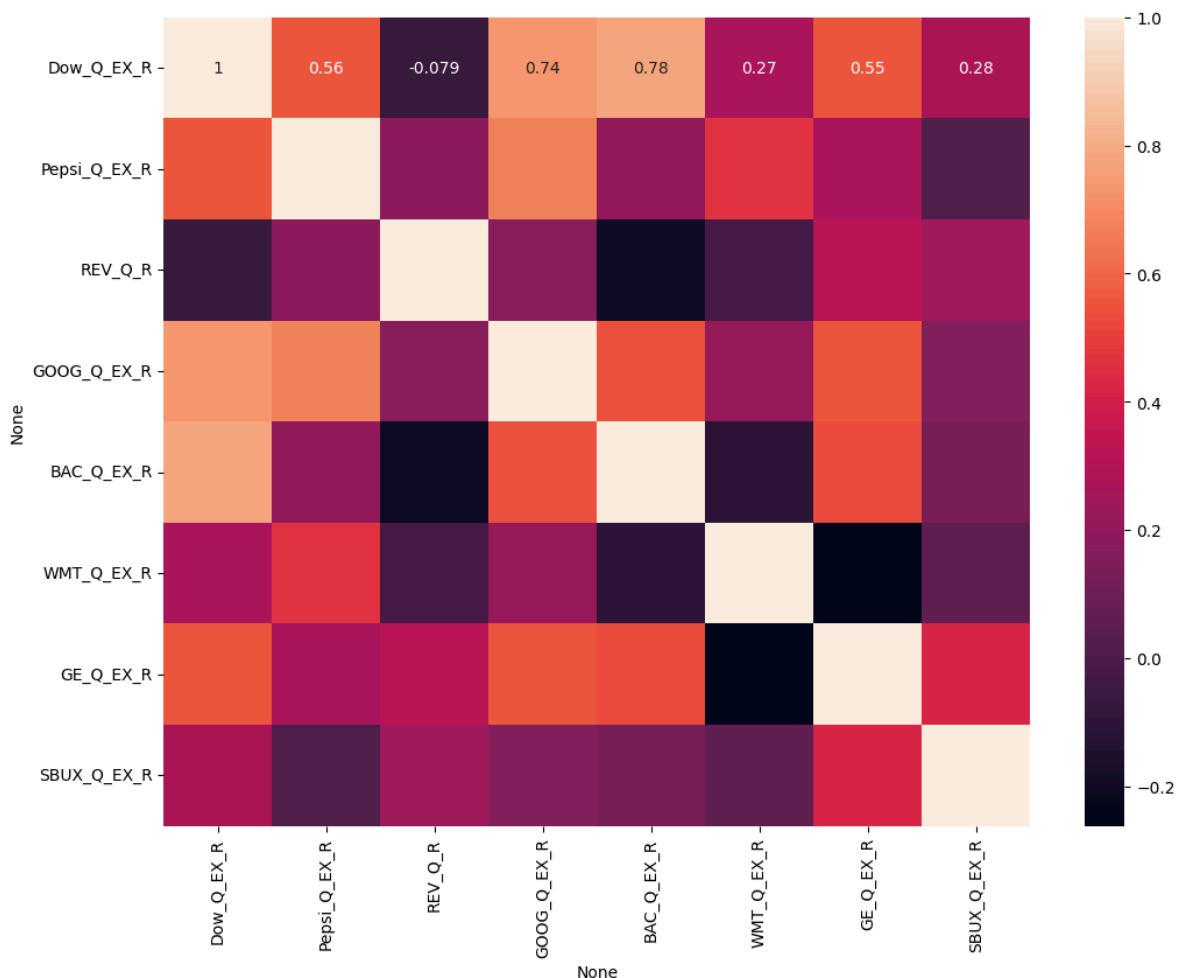
Adequacy Tests

Before going ahead with Factor analysis we need to verify whether the data is suitable to apply factor analysis concepts. For this we run the data through following tests:

- Correlation
- Bartlett's Test
- Kaiser-Meyer-Olkin (KMO) Test

Correlation - First step is to check correlation of all the variables. If some variables are highly correlated we will drop them.

```
In [15]: # Correlation plot of all the variables
c = df.corr()
sns.heatmap(c, annot=True)
plt.show()
```



From the above results, we observe, that we do have significant correlations between some of our variables in our example.

Bartlett's Test - Bartlett's test of sphericity checks whether or not the observed variables intercorrelate at all using the observed correlation matrix against the identity matrix. If the test found statistically insignificant, we should not employ a factor analysis. \$p\$-value less than 0.05 in our example below indicates statistically significant test.

```
In [5]: # Bartlett's Test
chi_square_value, p_value = calculate_bartlett_sphericity(df)
print("chi_square_value =", chi_square_value, "\np_value =", p_value)
```

```
chi_square_value = 81.69197183191892
p_value = 3.726332435213588e-07
```

Kaiser-Meyer-Olkin (KMO) Test - It is a statistic that indicates the proportion of variance in our variables that might be caused by underlying factors. High values (close to 1.0) generally indicate that a factor analysis may be useful with our data. If the value is less than 0.50, the results of the factor analysis probably won't be very useful.

```
In [6]: # Kaiser-Meyer-Olkin (KMO) Test
kmo_all, kmo_model = calculate_kmo(df)
print("kmo_model =", kmo_model)
```

```
kmo_model = 0.648886380028064
```

Performing factor analysis

Our data passed all adequacy test and we can go ahead with performing Factor Analysis. For choosing right number of factors we first need to take all 8 factors and find Eigenvalues.

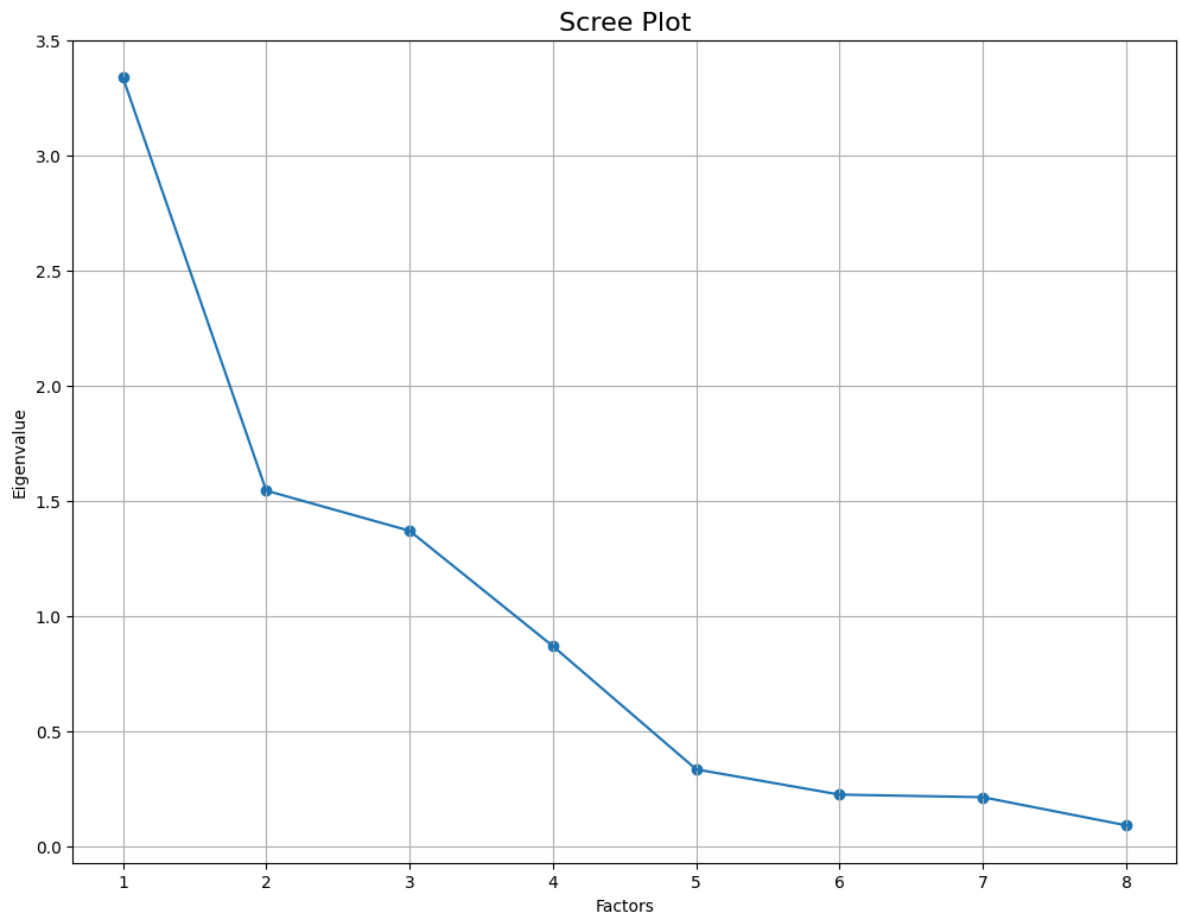
```
In [7]: # Create 8-factor object and fit data
fa1 = FactorAnalyzer(8, rotation=None)
fact_res = fa1.fit(df)

# Get Eigenvalues
ev, v = fact_res.get_eigenvalues()
ev
```

```
Out[7]: array([3.34031405, 1.54629398, 1.37192323, 0.8709355 , 0.33599621,
               0.22644586, 0.21514209, 0.09294909])
```

We can visualize Eigenvalues by plotting Scree plot:

```
In [8]: # Visualize eigenvalues on scree plot
plt.scatter(range(1, df.shape[1] + 1), ev)
plt.plot(range(1, df.shape[1] + 1), ev)
plt.title("Scree Plot", {"size": 16})
plt.xlabel("Factors")
plt.ylabel("Eigenvalue")
plt.grid()
plt.show()
```



How do we decide how many factors to consider in the final model? Results of eigenvalues and visualized Scree plot help us decide number of factors. In the Scree plot we can see that 3 eigenvalues are greater than 1 which in itself means that we need to chose 3-factor model.

The following code performs 3-factor analysis:

```
In [9]: # Create 3-factor object and fit data
fa3 = FactorAnalyzer(3, rotation=None)
fact_res3 = fa3.fit(df)

# Get uniquenesses
pd.DataFrame(
    fact_res3.get_uniquenesses(), index=[data.columns], columns=["uniquenesses"]
)
```

Out[9]:

	uniquenesses
Dow_Q_EX_R	0.079078
Pepsi_Q_EX_R	0.264248
REV_Q_R	0.520481
GOOG_Q_EX_R	0.280050
BAC_Q_EX_R	0.127822
WMT_Q_EX_R	0.541239
GE_Q_EX_R	0.004280
SBUX_Q_EX_R	0.837030

Let's look what the results tell us. The first section of the result is uniqueness. There is a number for each variable under uniqueness. This number is the ratio of unique variance over total variance of a variable, meaning how much of the total variance that this 3-factor model cannot explain. For Dow 30 excess return, the uniqueness is 0.079. It means there is only 7.9% of the total variance of the Dow 30 excess return that this 3-factor model cannot explain. It is a good result for the Dow 30 excess return. For Starbucks's excess return, the uniqueness is 0.837. It means that there is 83.7% of the variance of the Starbucks excess return that this 3-factor model cannot explain. This 3 factor model does a poor job for Starbucks's excess return. The ratio of the communality over the total variance of a variable is 1, uniqueness by definition. For example, Pepsi's ratio of communality over total variance is $1 - 0.264 = 0.736$. It means the 3-factor model can explain 74% of Pepsi's total variance.

Next, let's look at the loadings.

```
In [10]: # Get Factor Loadings
pd.DataFrame(
    fact_res3.loadings_,
    index=[data.columns],
    columns=["Factor 1", "Factor 2", "Factor 3"],
)
```

Out[10]:

	Factor 1	Factor 2	Factor 3
Dow_Q_EX_R	0.920076	0.118572	-0.245608
Pepsi_Q_EX_R	0.630328	0.532742	0.233719
REV_Q_R	0.130559	-0.105148	0.671876
GOOG_Q_EX_R	0.826906	0.169454	0.086381
BAC_Q_EX_R	0.735692	-0.257961	-0.514190
WMT_Q_EX_R	0.167506	0.653097	0.064548
GE_Q_EX_R	0.756477	-0.578518	0.297959
SBUX_Q_EX_R	0.280147	-0.190166	0.219827

The loading for Dow 30 excess return from factor 1 is 0.92. Factor 1 very well explains all the variance from Dow 30 excess return. Factor 1 also explains variances for Google's excess return and Bank of America's excess return pretty well. Factor 1 does poorly in explaining Walmart's and Starbucks's excess return variation.

For our example factor 1, Dow 30, Google and Bank of America have high loadings. For factor 2, Walmart and Pepsi have a high loading. For factor 3, KO's revenue growth has the highest loading.

Let's now look at Sum of Squared loadings and Proportional variances.

In [11]:

```
# Get proportion variance and cumulative proportion variance
pd.DataFrame(
    fact_res3.get_factor_variance(),
    index=["Sum of squared loadings", "Proportional variance", "Cumulative variance"],
    columns=["Factor 1", "Factor 2", "Factor 3"],
)
```

Out[11]:

	Factor 1	Factor 2	Factor 3
Sum of squared loadings	3.164713	1.201570	0.979488
Proportional variance	0.395589	0.150196	0.122436
Cumulative variance	0.395589	0.545785	0.668221

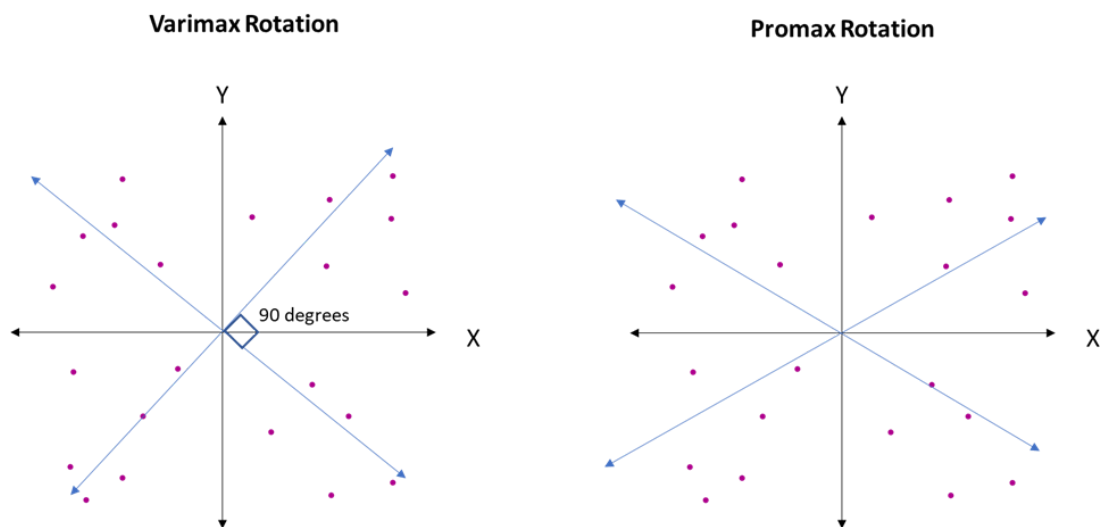
Proportional variance gives a ratio that indicates how much a factor explains the variance of all eight variables. For example, factor 1 explains 39.6% of the variance of the eight variables. Factor 2 explains 15% of the variance of the eight variables. Cumulative variance shows the cumulative ability of the factors to explain the variance of eight variables. For example, factor 1 and factor 2 combined explain 55% of the variance of eight variables. The 3-factor model explains a total of 67% of the variance of eight variables.

2.4 Rotation

One of the good things about factor analysis is that you can rotate the axes of the factors. Rotation usually makes only one factor highly associated with one or a small amount of initial variables. It simplifies the factor-variable structure and helps with interpretation of the factors. Rotation will not change the model results as to how much the model explains the total variance of the whole variables.

There are two types of rotation methods to choose from. The first one is called **varimax**. Varimax rotates the axes to make the factors uncorrelated. The second one is called **promax**. Promax rotation allows factors to be correlated. Figure 6 demonstrates the visual difference between the two rotation methods.

Figure 6: Varimax Rotation vs. Promax Rotation



You can try to run one model with one rotation method and run another model with another rotation method. You can then compare the loadings results and see which model result has a better interpretation. The following two figures show the results of the three-factor model with varimax rotation and the results of the three-factor model with promax rotation.

Performing factor analysis with factor rotation

The implementation of Factor Analysis with factor rotation element is done pretty much same manner as just Factor Analysis without rotation. Below is the example python code for Factor Analysis with Varimax rotation:

Figure 7: Results from Three-Factor Model with Varimax Rotation

```
In [12]: # Create 3-factor object with varimax rotation and fit data
fa3_varimax = FactorAnalyzer(n_factors=3, rotation="varimax", method="ml")
fact_res3_varimax = fa3_varimax.fit(df)
```

```

# Get Loadings
loadings_varimax = pd.DataFrame(
    fact_res3_varimax.loadings_,
    index=[data.columns],
    columns=["Factor 1", "Factor 2", "Factor 3"],
)

# Get factor variance
FactorVar_varimax = pd.DataFrame(
    fact_res3_varimax.get_factor_variance(),
    index=["Sum of squared loadings", "Proportional variance", "Cumulative variance"],
    columns=["Factor 1", "Factor 2", "Factor 3"],
)

# print the results
print("Loadings (varimax): \n", loadings_varimax)
print("\nFactor Variance (varimax): \n", FactorVar_varimax)

```

Loadings (varimax):

	Factor 1	Factor 2	Factor 3
Dow_Q_EX_R	0.844674	0.409035	0.169864
Pepsi_Q_EX_R	0.265763	0.731598	0.269637
REV_Q_R	-0.222475	0.094626	0.576091
GOOG_Q_EX_R	0.589017	0.474461	0.357928
BAC_Q_EX_R	0.951142	-0.060817	-0.007843
WMT_Q_EX_R	-0.049185	0.754406	-0.166020
GE_Q_EX_R	0.556886	-0.132580	0.806556
SBUX_Q_EX_R	0.166399	0.020042	0.400718

Factor Variance (varimax):

	Factor 1	Factor 2	Factor 3
Sum of squared loadings	2.425442	1.527419	1.400282
Proportional variance	0.303180	0.190927	0.175035
Cumulative variance	0.303180	0.494108	0.669143

Figure 8: Results from Three-Factor Model with Promax Rotation

```

In [13]: # Create 3-factor object with promax rotation and fit data
fa3_promax = FactorAnalyzer(n_factors=3, rotation="promax", method="ml")
fact_res3_promax = fa3_promax.fit(df)

# Get Loadings
loadings_promax = pd.DataFrame(
    fact_res3_promax.loadings_,
    index=[data.columns],
    columns=["Factor 1", "Factor 2", "Factor 3"],
)

# Get factor variance
FactorVar_promax = pd.DataFrame(
    fact_res3_promax.get_factor_variance(),
    index=["Sum of squared loadings", "Proportional variance", "Cumulative variance"],
    columns=["Factor 1", "Factor 2", "Factor 3"],
)

```

```
# print the results
print("Loadings (promax): \n", loadings_promax)
print("\nFactor Variance (promax): \n", FactorVar_promax)
```

Loadings (promax):

	Factor 1	Factor 2	Factor 3
Dow_Q_EX_R	0.838854	0.295139	-0.019440
Pepsi_Q_EX_R	0.149968	0.696324	0.184077
REV_Q_R	-0.381838	0.072336	0.663141
GOOG_Q_EX_R	0.510798	0.378787	0.232134
BAC_Q_EX_R	1.045865	-0.188392	-0.199542
WMT_Q_EX_R	-0.098602	0.803417	-0.225000
GE_Q_EX_R	0.441448	-0.288332	0.769343
SBUX_Q_EX_R	0.089687	-0.039682	0.398610

Factor Variance (promax):

	Factor 1	Factor 2	Factor 3
Sum of squared loadings	2.439357	1.486367	1.369125
Proportional variance	0.304920	0.185796	0.171141
Cumulative variance	0.304920	0.490716	0.661856

From Figure 7 and Figure 8, we can compare the three-factor model with varimax or with promax. The model results of the two different rotations are very close. Their results are also close to the results with no rotation. In this example, we don't need rotation to help with factor interpretation.

2.5 Factor Analysis vs. Principal Component Analysis

A lot of people often confuse factor analysis and principal component analysis. Both factor analysis and principal component analysis are dimension reduction methods. Factor analysis has factors, and principal component analysis has principal components. So what is the difference between the two methods?

Principal components from PCA are generated from a linear combination of the variables. The purpose of PCA is to find principal components to explain as much of the total variance of the original variables as possible. Factor analysis is a model to measure factors or latent variables. The purpose of factor analysis is to find latent variables that can best explain the covariance among original variables, not the total variance of the dataset.

3. Conclusion

In this lesson, we first introduced measurement error. We discussed the implications of having measurement error in one or several variables in a linear regression model. We then moved to study factor analysis, especially exploratory factor analysis. We used exploratory factor analysis to measure latent variables and model measurement error. Both factor analysis and principal component analysis are used to reduce dimension for the dataset. We briefly talked about the difference between the two methods.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. "Multiple Comparisons", Eberly College of Science, Penn State University

<https://online.stat.psu.edu/stat503/lesson/3/3.3>

- **Required pages:** Bonferroni Method
- **Estimated reading time:** 15 minutes

2. "Conditional Distributions", Eberly College of Science, Penn State University

<https://online.stat.psu.edu/stat505/lesson/6/6.1>

- **Required pages:** Whole webpage
- **Estimated reading time:** 60 minutes

FINANCIAL ECONOMETRICS

MODULE 7 | LESSON 2

NETWORK THEORY

Reading Time 60 minutes

Prior Knowledge OLS, Covariance Matrix, Penalized LASSO Regression, Basic Python

Keywords | Network Theory, Graph Theory, Adjacency Matrix, Directed Network, Undirected Network, Bonferroni Correction, Partial Correlation Matrix, Precision Matrix, Graphical LASSO
|||

1. Introduction

In this lesson, we are going to introduce a modern analytical approach to understand the interactions among various financial entities. It is called network theory. Specially, we will incorporate regression analysis and time series we learned in the previous lessons into network theory analysis. We will first learn the basics of network theory. Then we will apply various regression models to construct a network. We will finish the lesson by learning a more general and efficient method to build network.

2. The Basics of Network Theory

2.1 What is Network Theory

Network Theory is an analytical method to study relationships and interactions among components in a system. Network Theory is also called Graph Theory in Mathematics. Although there are technical differences between network theory and graph theory but we will not discuss these differences in this lesson. Network theory can effectively analyze the complex interactions among components. Swiss Mathematician, Leonard Euler, is credited to start using this approach to solve the Königsberg bridge problem. The problem statement is to find a path to cross 7 bridges in Königsberg without crossing any bridge twice as in the following figure 0(Paoletti, 2011).

Figure 0. Seven Bridges of Königsberg



M7_L1_seven_bridges Source: Europe Between East and West

Initially, network theory was mainly used in social network studies. Today, it has been widely applied in various financial research, notably portfolio management and risk management. Network theory provides insights on how financial assets interact with each other with different scenarios or how financial entities respond during risk events. One application is to analyze the systematic risk among financial institutions. In a financial institution network, there are banks, investors and other financial market participants. They are all interconnected through trading, investment, transfer and other financial linkages. By using network theory, researchers can understand the degree of interconnections among all players, identify key hubs in the system and even simulate how fast and wide a risk factor can spread throughout the whole network.

Please keep in mind that this section is meant to introduce network theory. You will see network theory extensively covered in courses to come: Stochastic Modeling, Machine Learning in Finance, Portfolio Management, and Risk Management. In the following sections, we will learn the very basics of network theory. Our focus will then turn to incorporate statistical methods we have learned from this course into network theory. We will explore Network theory in great depth in portfolio management and risk management courses.

2.2 Network Structure

A network is a **complex system**. In a complex system, it consists of two main types of elements: **Components** and **Interactions**. A component can also be called a **Node** or a **Vertex**. An interaction is also called a **Link** or a **Edge**. The following figure illustrates a basic network structure.

Figure 1. Illustration of A Simple Network Structure



Figure 1 provides a simple representation of a network structure. In a financial institutions network, nodes might represent commercial banks, individual investors, regulatory bodies, and other financial market participants. The edges represent activities through which these entities interact. For example, if node C is a commercial bank and node B is an individual investor, the edge between them might represent deposit activity from B to C. If node F is a clearing system, the edge between nodes C and F represents settlement of money transfers between C and other commercial banks in the network. In this figure, we have 7 nodes and 8 edges. We typically use **N** to represent the total number of nodes in a network and **L** to represent the total number of edges. In a network, interactions can be one-directional. For example, an individual (node B) might deposit money with a commercial bank (node C), but there may not be an equivalent reverse interaction. We can use arrows to demonstrate these directional relationships.

Figure 2. Illustration of A Directed Network Structure



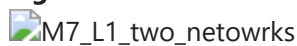
Figure 2 shows the interaction flow among all nodes with lines with arrows. Arrows indicate the direction of the interaction flow. This type of network with arrow links in Figure 2 is called a **Directed Network**. In contrast, figure 1 represents an **Undirected Network**, where the links do not have arrows. For an undirected network, the relationships between nodes are reciprocal or bidirectional. For example, the interaction flow can go from node B to node C as well as node C to node B. Because of this feature, an undirected network is also **symmetrical**.

For a directed network, there are arrows on the links or edges. **The arrows can be one-directional or bidirectional**. The links in a directed network are also called **arcs**.

2.3 Adjacency Matrix

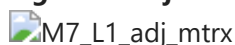
A common method to represent a network in data form is to use an **Adjacency Matrix**. This approach describes a network structure in a matrix. Let's examine some examples of adjacency matrix. In figure 3, there are two networks. One is an undirected network and the other one is directed network.

Figure 3. Undirected Network v.s. Directed Network



In figure 3, these two networks all have 4 nodes and 4 links. The adjacency matrices for the two networks are in the following figure 4.

Figure 4. Adjacency Matrices for Undirected Network and Directed Network



Let's look at the undirected network matrix on the left first. The node names are labeled along the top and the left side of the matrix for clarity. In an adjacency matrix, A_{ij} indicates an adjacency matrix with row i and column j . The values are defined as follows:

$A_{ij} = 1$ if a link pointing from node j to node i exists

$A_{ij} = 0$ otherwise

When reading an adjacency matrix, we would start from columns first. For an undirected network's adjacency matrix, the values in column A indicate if there is a link pointing from node A to node B, node C or node D. In the undirected network in figure 4, the links from node A to node B, node C and node D are two-way. Hence the values are 1s. Also in this network, node A does not have a link to itself so the value from node A to node A is 0. Therefore, the values of column A are (0,1,1,1).

Looking at column D in the undirected network, the only link that exists is the link pointing from node D to node A. Hence the first entry of the column D is 1. Since there are no other link for node D, the rest of the entries in column D are 0s. Therefore, the values of column D are (1,0,0,0). With the same logic, we can fill in the entries of adjacency matrix for directed network.

Some observations about the adjacency matrix:

- The diagonal values of the adjacency matrix for an undirected network are all 0s if every node does not have a link to themselves.
- The adjacency matrix for an undirected network is **symmetric**.
- An adjacency matrix is usually **sparse**. It is because each node would only have a few links to some nodes in the network. That means the adjacency matrix is a sparse matrix.
- When all nodes in a network connect to all other nodes, this network is called a **complete graph**.

The adjacency matrices shown above have entries of only 0s and 1s, representing the presence or absence of connections. These are called **Unweighted Adjacency Matrices**. We can relax this assumption and allow connected nodes to have various strength of connections. When the strength of connections among nodes in a network are not only 1s or 0s, this network has a **Weighted Adjacency Matrix**. Figure 5 shows an example of the weighted adjacency matrix for a directed network.

Figure 5. Illustration of A Weighted Adjacency Matrix

 M7_L1_weighted_amtrx

In the weighted adjacency matrix in figure 5, the weights represent how strong nodes connected to each other rather than just mere having connection or not. The weights can be positive or negative. How should we explain these weights? Suppose A,B,C,D,E represent different banks in the network. The link means the money transfer from one bank to another bank. The weight is the average amount of money transfer happens in a day from one bank to the other. In this case, bank B transfers money to bank A on average 5 times a day. But bank A does not transfer money to bank B.

To understand the relationships among nodes in a directed network is critical for many financial analyses. The analysis usually involves determining the weights for the adjacency matrix. In the subsequent section, we will use statistical methods we learned from previous lessons to find these weights for the adjacency matrix of a directed network.

3. Regression-Based Models for Asset Networks

In this section, we are going to combine regression models with network theory to understand the interactions among financial assets and other factors. We will use the asset network framework provided by Konstantinov and Fabozzi(2025) to conduct the analysis.

3.1 Overview

In financial network, the nodes are usually known, such as asset prices, financial institutions, economic factors etc. However the links/edges are not known. We do not know the relationship between nodes but we would like to build models to explain the interactions. There are several methods to model the relationship. One of the methods is to use statistical models. These models usually are developed using time series data. The most common type of statistical models are regression models. The goal of regression models is to find relationship between financial assets and present the relationship in a network form.

Regression models establish direct relationships between dependent and independent variables. This feature provides a straightforward approach to model links between network nodes. In financial networks, these nodes typically represent different financial assets. One key reason to use regression models for interactions is that regression coefficients serve as indicators of association between variables. These coefficients help determine which variables are connected to the dependent variable and the nature of their relationship. The significance of regression parameters plays a crucial role in determining entries in the adjacency matrix. Understanding this relationship between regression coefficients and network connections is essential for properly interpreting regression analysis results in the context of network construction.

3.2 Data

As we mentioned before, the statistical models for network theory usually use time series data for analysis. For the following regression models for asset networks, we will use the daily close prices of the following stocks from 2018 to 2024. This period includes a few US stock market cycles so that the data can be representative of different stages of the stock market. Here are the 7 stocks prices included in the dataset:

- Apple (AAPL)
- Walmart (WMT)
- Amazon (AMZN)
- General Electric (GE)
- Google (GOOGL)
- JP Morgan (JPM)
- Warner Brothers (WBD)

We will also include the following stock indices or other financial asset data as factors for analysis:

- S&P 500 index (^GSPC)
- Financial Select Sector SPDR Fund (XLF)
- Technology Select Sector SPDR Fund (XLK)
- Short-term Treasury ETF (SHY)
- US Dollar index (DX-Y.NYB)

We will then convert the above prices and indices to returns. From previous lessons, we know returns possess many benefits for analysis. First, let's import some Python packages and gather data for analysis.

```
In [1]: pip install networkx
```

Requirement already satisfied: networkx in /usr/local/lib/python3.11/site-packages (3.3)

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>. Use the --root-user-action option if you know what you are doing and want to suppress this warning.

Note: you may need to restart the kernel to use updated packages.

```
In [2]: pip install seaborn --upgrade
```

Requirement already satisfied: seaborn in /usr/local/lib/python3.11/site-packages (0.13.2)

WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x743bf665f990>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x743bf6664c10>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

WARNING: Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x743bf6665890>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

WARNING: Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x743bf6666610>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

WARNING: Retrying (Retry(total=0, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x743bf6667210>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.11/site-packages (from seaborn) (2.1.3)

Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.11/site-packages (from seaborn) (2.2.3)

Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.11/site-packages (from seaborn) (3.10.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.56.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.1.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.1)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/site-packages (from pandas>=1.2->seaborn) (2025.1)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/site-packages (from pandas>=1.2->seaborn) (2025.1)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>. Use the --root-user-action option if you know what you are doing and want to suppress this warning.

Note: you may need to restart the kernel to use updated packages.

```
In [3]: import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from statsmodels.formula.api import ols
from sklearn.covariance import GraphicalLassoCV
from sklearn.covariance import GraphicalLasso
from sklearn.decomposition import PCA
```

```
In [4]: # Gather stock data and factor data for 7 years

# List of assets (stocks)
assets = ['AAPL',    # Apple
          'WMT',     # Walmart
          'AMZN',    # Amazon
          'GE',      # General Electric
          'GOOGL',   # Google
          'JPM',     # JP Morgan
          'WBD']     # Warner Brothers

# List of factors
factors = [
    '^GSPC',    # S&P 500 (market factor)
    'XLF',      # Financial Select Sector SPDR Fund
    'XLK',      # Technology Select Sector SPDR Fund
    'SHY',      # Short-term Treasury ETF (interest rate factor)
    'DX-Y.NYB'  # US Dollar index
]

# Combine assets and factors
tickers = assets + factors

# Download data
data_0 = pd.read_csv('M7L2_stock_prices.csv')

data = data_0.drop('Date', axis=1)

# Calculate returns
returns = data.pct_change().dropna()

# Add meaningful labels
returns.columns = [
    'Apple', 'Amazon', 'USD', 'GE', 'Google', 'JPMorgan', 'Treasury', 'Warner-Bro',
]
```

4. Simple Linear Regression Model for Asset Network

In this section, we will start by using simple linear regression (OLS) to model the interaction of 6 stock returns (nodes) to construct an asset network. This asset network will be a

bidirectional network, which means the relationship can be reciprocal between any two nodes. For stock i and stock j , we will have one regression that returns for stock i will be the dependent variable and returns of stock j will be the independent variable. But we will also have another regression that returns for stock j will be the dependent variable and returns for stock i will be independent variable. The following regressions demonstrate the two regressions for stock i and stock j .

$$R_i = \alpha_{ij} + \beta_{ij}R_j + \varepsilon_{ij}(1) \\ R_j = \alpha_{ji} + \beta_{ji}R_i + \varepsilon_{ji}(2)$$

Let's look at the regression (1) first. From the standard OLS regression explanation, β_{ij} coefficient means when R_j increases by one unit, R_i will increase by β_{ij} units. If the estimate of β_{ij} coefficient is statistically significant, we can say this regression relationship exists for R_i and R_j . In the asset network context, β_{ij} shows the tendency that R_i can be influenced by R_j . If β_{ij} is statistically significant, then the influence exists. We can say that R_i responds to R_j , or the link goes from R_j to R_i ($R_j \rightarrow R_i$). Although the above equations look like CAPM, but there are significant differences. In CAPM, the independent variable is a market proxy variable. But here we have an individual stock return as the independent variable.

For regression (2), if β_{ji} is statistically significant, we can infer that R_j responds to R_i . The link between the two nodes goes from R_i to R_j ($R_i \rightarrow R_j$).

Our next step is to check p value of β_{ji} coefficient for statistical significance. In our simple network model with only two assets, we have two regression models. We have to conduct two tests simultaneously to check for significance of β_{ij} and β_{ji} . Usually we would set our p value threshold at 0.05 for a test. If the p value of β_{ij} is less than 0.05, then β_{ij} is statistically significantly different from 0. However, we have two tests in this case. We will need to adjust p value to avoid the multiple testing problem.

The multiple testing problem arises when conducting several statistical tests simultaneously without adjusting p-values, leading to an inflated risk of false positives. When using the p value threshold of $\alpha = 0.05$ for a single test, you accept a 5% chance of incorrectly rejecting the null hypothesis. However, this error rate compounds dramatically across multiple tests. For example, when conducting 5 independent tests at $\alpha = 0.05$, the probability of obtaining at least one false positive jump to approximately 23% ($1 - (1 - 0.05)^5 = 0.226$). There are many methods to adjust p value for multiple tests. In this lesson, we will use Bonferroni method. Please review the required reading to learn more about Bonferroni method. The way Bonferroni method adjusts p value threshold is to divide original threshold (say 0.05) by the number of tests conducted. In the simple two asset network case, the adjusted p value threshold would be $0.05/2 = 0.025$. In our 6-stock returns network, the adjusted p value threshold will be $0.05/30 = 0.00167$. (Why 30? This question will leave as an exercise for

readers to find out).

If we confirm that β_{ij} is statically significant, we can assert the following link exists: $R_j \rightarrow R_i$. We can also fill in the entry in the adjacency matrix using the following rule:

$$A_{ij} = 1 \text{ if } \beta_{ij} \text{ is statistically significant}$$
$$A_{ij} = 0 \text{ otherwise}$$

With the filled adjacency matrix, we can draw the network and investigate the relationship among asset returns in the asset network. The following Python code implements the steps we describe in this section with the visualization of the asset network.

```
In [5]: # Create a dataframe that only includes assets
assets = returns[["Apple", "Amazon", "GE", "Google", "JPMorgan", "Warner-Bro", "Walmart"]

# Run a Simple Linear Regression Based Asset Network
n_assets = len(assets.columns)
adj_matrix = np.zeros((n_assets, n_assets))

# Significance threshold (adjusted based on Bonferroni Method)
p_threshold = 0.05/(n_assets*(n_assets-1))

# Loop through all pairs of assets
for i, asset_i in enumerate(assets):
    for j, asset_j in enumerate(assets):
        if i != j: # Skip self-relationships
            # Run regression: asset_i ~ intercept + asset_j
            model = sm.OLS(assets[asset_i], sm.add_constant(assets[asset_j])).fit()

            # Get p-value for the slope coefficient
            p_value = model.pvalues.iloc[1]

            # If significant, add connection to adjacency matrix
            adj_matrix[i, j] = 1 if p_value < p_threshold else 0

# Create directed graph from adjacency matrix
G = nx.from_numpy_array(adj_matrix, create_using=nx.DiGraph)

# Relabel nodes with asset names
mapping = {i: name for i, name in enumerate(assets)}
G = nx.relabel_nodes(G, mapping)

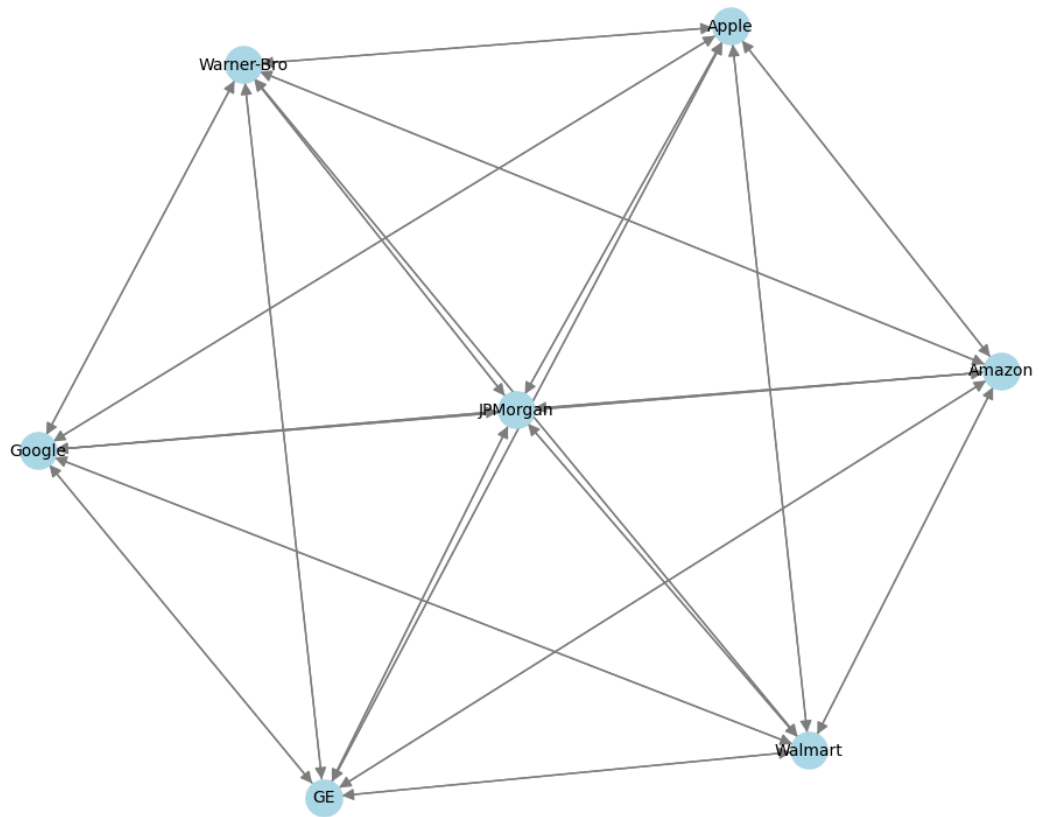
# Visualize the network
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos,
        node_color='lightblue',
        node_size=500,
        with_labels=True,
        font_size=10,
        edge_color='gray',
        arrows=True,
```

```

        arrowsize=15)
plt.title('Simple Linear Regression-Based Asset Network')
plt.show()

```

Simple Linear Regression-Based Asset Network



In [6]: adj_matrix

```

Out[6]: array([[0., 1., 1., 1., 1., 1., 1.],
               [1., 0., 1., 1., 1., 1., 1.],
               [1., 1., 0., 1., 1., 1., 1.],
               [1., 1., 1., 0., 1., 1., 1.],
               [1., 1., 1., 1., 0., 1., 1.],
               [1., 1., 1., 1., 1., 0., 1.],
               [1., 1., 1., 1., 1., 1., 0.]])

```

In the above simple linear regression based asset network, we can see all stock returns are connected to other stock returns in this network. This is a directed network. From the graph the relationship between either of the two asset returns are all bidirectional. We can also print out the adjacency matrix to confirm the finding. This linear regression based asset network does not seem to provide much information. We have to try another regression model structure. We will try non-linear regression based asset network.

5. Non-Linear Regression Model for Asset Network

In this section, we test non-linear regression based asset network. We learned that linear regression based asset network does not provide much information about the relationship among 6 stock returns. One way we can adjust the regression model is to add a squared independent variable to make the regression non-linear. Following the two-stock returns example from the last section, the regressions can be adjusted as follows:

$$\begin{aligned} R_i &= \alpha_{ij} + \beta_{ij}R_j + \gamma_{ij}R_j^2 + \epsilon_{ij}(3) \\ &= \alpha_{ji} + \beta_{ji}R_i + \gamma_{ji}R_i^2 + \epsilon_{ji}(4) \end{aligned}$$

With the adjustment of the regressions, our focus will still be β to understand if the linear part of the relationships between stock returns exists. The following Python code implements the non-linear regression based asset network model for 6 stock returns.

```
In [7]: # Run a Non-Linear Regression Based Asset Network
n_assets = len(assets.columns)
adj_matrix = np.zeros((n_assets, n_assets))

# Significance threshold (can be adjusted)
p_threshold = 0.05/(n_assets*(n_assets-1))

# Loop through all pairs of assets
for i, asset_i in enumerate(assets):
    for j, asset_j in enumerate(assets):
        if i != j: # Skip self-relationships
            # Create squared term
            indep = pd.concat([assets[asset_j], assets[asset_j]**2], axis = 1)
            # Run regression with intercept, original term, and squared term
            X = sm.add_constant(indep)
            model = sm.OLS(assets[asset_i], X).fit()

            # Get p-value for the slope coefficient
            p_value = model.pvalues.iloc[1]

            # If significant, add connection to adjacency matrix
            adj_matrix[i, j] = 1 if p_value < p_threshold else 0

# Create directed graph from adjacency matrix
G = nx.from_numpy_array(adj_matrix, create_using=nx.DiGraph)

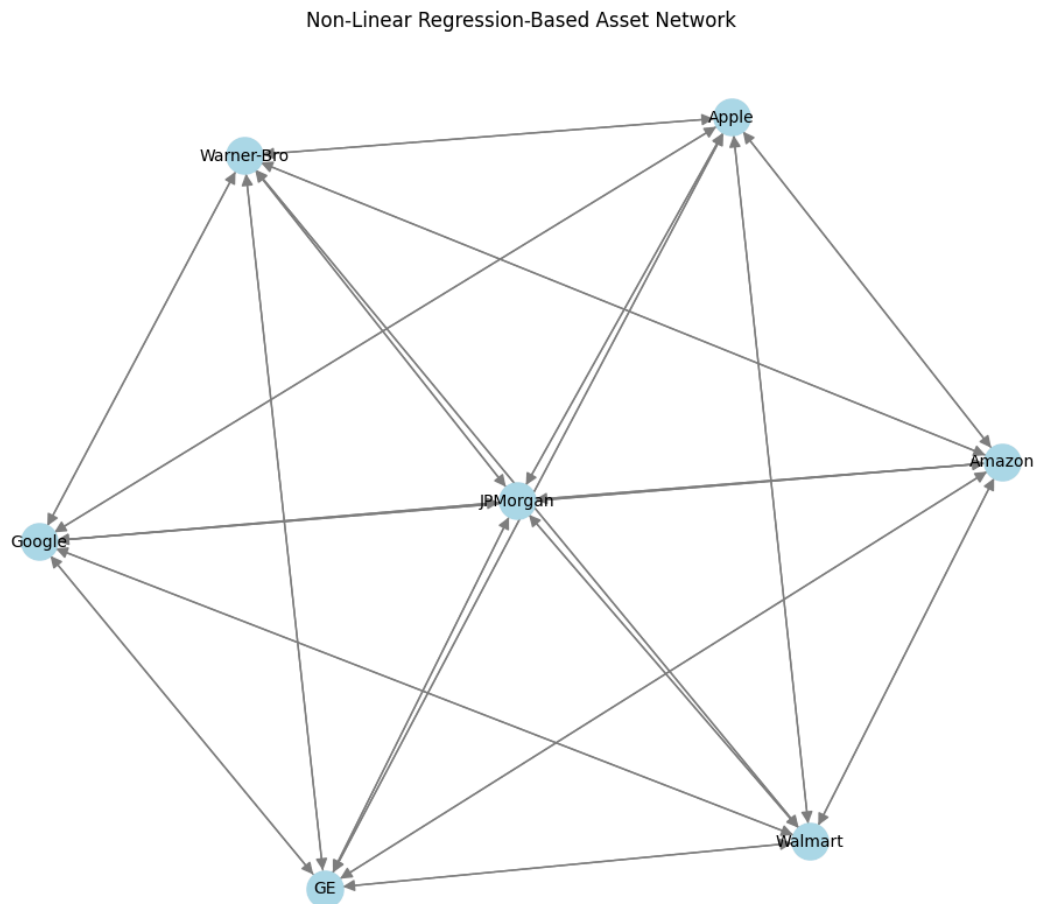
# Relabel nodes with asset names
mapping = {i: name for i, name in enumerate(assets)}
G = nx.relabel_nodes(G, mapping)

# Create and visualize the network
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos,
        node_color='lightblue',
```

```

node_size=500,
with_labels=True,
font_size=10,
edge_color='gray',
arrows=True,
arrowsize=15)
plt.title('Non-Linear Regression-Based Asset Network')
plt.show()

```



In [8]: adj_matrix

```

Out[8]: array([[0., 1., 1., 1., 1., 1., 1.],
               [1., 0., 1., 1., 1., 1., 1.],
               [1., 1., 0., 1., 1., 1., 1.],
               [1., 1., 1., 0., 1., 1., 1.],
               [1., 1., 1., 1., 0., 1., 1.],
               [1., 1., 1., 1., 1., 0., 1.],
               [1., 1., 1., 1., 1., 1., 0.]])

```

By reviewing the above graph result from the non-linear regression based asset network, we do not see difference from the graph from linear regression models. Clearly, using squared returns of stocks does not improve the return of the stock; so this non-linearity is not helping the regression. In the next section, we will introduce a new term to the regression models to see if this new adjustment can provide more insights into the asset network.

6. Linear Regression Model with a Confounder for Asset Network

Now, let's think about our two-stock asset network a little deeper. In the previous sections, we only use the returns of these two stocks to identify their relationship. Sometimes, that model set up may not have enough economic connections. There can be a case where the reason the two stocks are correlated is because they are all influenced by a common factor not in the model. In our case, an equity index may be the common factor influencing both stock returns. Therefore, incorrect regression model specification without well-thought economic connections can produce false variable relationships that actually do not exist.

To handle the issue of overfitting or false relationship generated from regression models, researchers have proposed to include factors as confounders in the models (López de Prado, 2003). The key criteria to select a confounder is that the confounder should be correlated to an independent variable(s). The confounder should also causally relate to dependent variable since we are analyzing a directed asset network. Say R_C is the return of the confounder variable, we can include R_C in our regression models as follows:

$$R_i = \alpha_{ij} + \beta_{ij}R_j + \rho_{ij}R_C + \epsilon_i \quad (5)$$
$$R_j = \alpha_{ji} + \beta_{ji}R_i + \rho_{ji}R_C + \epsilon_j \quad (6)$$

The choice of a confounder depends on econometric analysis. In our case, since our assets are all stocks, S&P 500 index is a good option as a confounder as it represents the general stock market exposure.

We can rerun our linear regression models with S&P 500 returns and review the result. We are still focusing on β estimates from the regression models.

```
In [9]: # Run a Linear Regression Based Asset Network with a Confounder
n_assets = len(assets.columns)
adj_matrix = np.zeros((n_assets, n_assets))
SP500 = returns["SP500"]

# Significance threshold (can be adjusted)
p_threshold = 0.05/(n_assets*(n_assets-1))

# Loop through all pairs of assets
for i, asset_i in enumerate(assets):
    for j, asset_j in enumerate(assets):
        if i != j: # Skip self-relationships
            # Create squared term
            indep = pd.concat([assets[asset_j], SP500], axis = 1)
            # Run regression with squared term
            X = sm.add_constant(indep)
            model = sm.OLS(assets[asset_i], X).fit()
```

```

        # Get p-value for the slope coefficient
        p_value = model.pvalues.iloc[1]

        # If significant, add connection to adjacency matrix
        adj_matrix[i, j] = 1 if p_value < p_threshold else 0

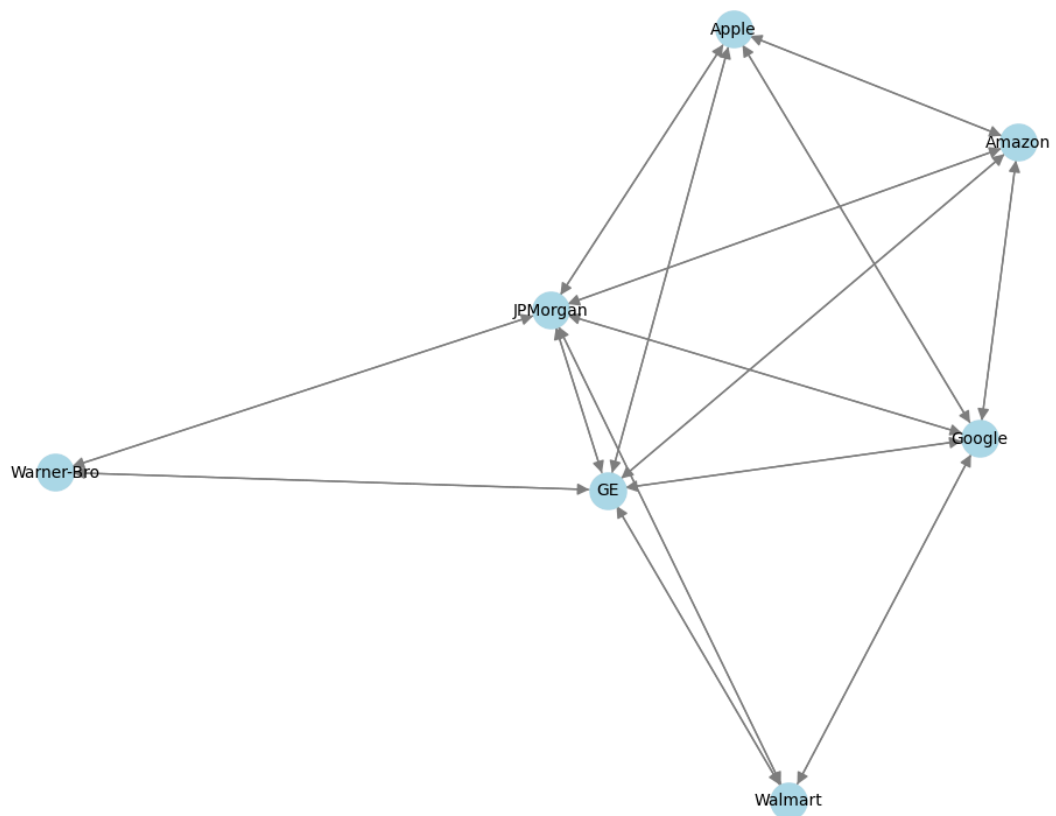
# Create directed graph from adjacency matrix
G = nx.from_numpy_array(adj_matrix, create_using=nx.DiGraph)

# Relabel nodes with asset names
mapping = {i: name for i, name in enumerate(assets)}
G = nx.relabel_nodes(G, mapping)

# Create and visualize the network
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos,
        node_color='lightblue',
        node_size=500,
        with_labels=True,
        font_size=10,
        edge_color='gray',
        arrows=True,
        arrowsize=15)
plt.title('Linear Regression-Based Asset Network with S&P500 as Confounder')
plt.show()

```

Linear Regression-Based Asset Network with S&P500 as Confounder



By controlling the confounder variable in our regression models, the new asset network reveals some interesting information. From the graph, we can see that JPMorgan and GE are the only two which have bidirectional relationships to all other stocks. JPMorgan and GE can be treated as two hubs in this asset network. We all know that JPMorgan is a financial institution so it should have business relationship with other companies. As for GE, it also has a financing division in its business and can be treated as a financial institution too. The next interesting observation is all three technology stocks (Apple, Amazon and Google) have bidirectional relationship too. It shows stocks in technology section can be highly interconnected. The last interesting observation to point out is that Warner Brothers stock return and Walmart's stock returns are not connected after including S&P 500 index return to the model. It says that both stock returns are not directly correlated after taking into account of S&P500 index return. We say Warner Brothers stock returns and Walmart's stock returns are **conditionally independent**. The conditional independence also happens to other pairs.

So far, we have been using regression analysis to model the links between nodes in a network. When conducting regression analysis under network framework, we still need to pay attention to the following items.

- check for linearity for the dataset

- outliers in the dependent variables
- leverage points in the independent variables
- heteroscedasticity of error terms
- multicollinearity between independent variables and the confounder

7. Asset and Factor LASSO Model

7.1 Overview

The previous section demonstrated the power of adding a confounder variable to an asset network. Now we face a new challenge: what if multiple potential confounders exist, all with economic justification? We need a more efficient method to test these confounders in our regression models when estimating network structure. In this section, we'll refer to these confounder variables as **Factors** and introduce **Graphical LASSO** (Graphical Least Absolute Shrinkage and Selection Operator).

Our goal remains to identify direct relationships between assets while controlling other factors. These direct relationships form the foundation of our asset and factor network. **Partial correlation** is the appropriate measure for evaluating these direct relationships. Unlike standard correlation, partial correlation quantifies the relationship between two variables while controlling the effects of other variables. This approach helps identify genuinely direct connections between assets, filtering out indirect relationships that occur through intermediary assets, thus providing a clearer picture of the underlying network structure in financial markets. Please review the required reading "Conditional Distributions" for more information about partial correlation.

In the network context, we will need to have a partial correlation matrix to describe partial correlations for all assets and factors in the network. From there we can build links among nodes. In the real financial market, not all assets and factors are all partially correlated. Many of the partial correlations can be 0s. To translate this observation to partial correlation matrix, we can say partial correlation matrix is **sparse**. In order to identify sparsity in the partial correlation matrix, we can use LASSO or L1 penalty to impose constraints while estimating partial correlation matrix. The lasso penalty enforces this sparsity by shrinking many partial correlations exactly to zero. When the partial correlation of two variables 0 we say these two variables are conditionally independent.

7.2 Graphical LASSO Model

In a Graphical LASSO model, there are p random variables in a dataset $\mathbf{X}$ follows a multivariate normal distribution as follows: $\mathbf{X} = (X_1, X_2, \dots, X_p) \sim \text{MVP}(\mu, \Sigma)$

Let's define $\Theta = \Sigma^{-1}$, where Θ is the inverse of the covariance matrix. Θ is called **Precision Matrix**. The elements of this precision matrix are proportional to the partial correlation matrix. By estimating the precision matrix, we can determine the partial correlations needed to construct our network.

To incorporate the sparse nature of partial correlation matrices in asset and factor networks, we apply a LASSO penalty to the model. Friedman et al. (2008) proposed an efficient method for solving the inverse covariance matrix in a Graphical LASSO model by maximizing the following penalized log-likelihood function with respect to the mean vector μ :

$$-\log \left\{ \det(\Theta) - \text{trace}(S\Theta) - \rho \|\Theta\|_1 \right\}$$

where

Θ is the inverse of covariance matrix

S is the sample covariance matrix

ρ is the penalty parameter to control sparsity

$\|\Theta\|_1$ is the L1 norm—the sum of the absolute values of the elements of Θ

The parameter ρ is crucial: increasing ρ produces a sparser network by forcing more elements of Θ to zero, which in turn makes the corresponding partial correlations zero. Selecting an appropriate value for ρ requires careful consideration. If ρ is too high, we'll have no connections among assets; if ρ is too low, all assets will be connected, rendering the network uninformative. In our application, we'll employ cross-validation to determine the optimal ρ .

7.3 Application

In this section, we'll apply the Graphical LASSO model to the dataset described in Section 3.2 to construct an asset and factor network. We'll utilize all 7 stock returns we used in the previous sections. We will also add S&P500 ETF, Treasury ETF, Financial sector ETF, Technology ETF and US Dollar Index as factors to the model. This model will demonstrate how the precision matrix can reveal the underlying structure of financial markets while controlling common factors.

First, let's standardize all the stock and factor return data.

```
In [10]: # Standardize data
standardized_returns = (returns - returns.mean()) / returns.std()
```

Next, we will use cross validation method to find the optimal level of ρ for analysis.

```
In [11]: # Create a range of rhos to test
rhos = np.linspace(0.3, 1, 100)
```

```

# Use GraphicalLassoCV to find the best alpha
model_cv = GraphicalLassoCV(alphas=rhos, cv=5, tol=1e-2, max_iter=400)
model_cv.fit(standardized_returns)

#best_rho = 0.3
best_rho = model_cv.alpha_
print(f"Best p: {best_rho}")

```

Best p: 0.3

With the optimal ρ available, we can now fit the graphical LASSO model and estimate the inverse of covariance matrix. Then we can calculate partial correlation matrix.

```

In [12]: # Fit the model with the best alpha
model = GraphicalLasso(alpha=best_rho, max_iter=400, tol=1e-2)
model.fit(standardized_returns)

# Extract precision matrix (inverse covariance)
precision_matrix = model.precision_

# Convert precision matrix to partial correlation matrix for easier interpretation
d = np.sqrt(np.diag(precision_matrix))
partial_corr = -precision_matrix / np.outer(d, d)
np.fill_diagonal(partial_corr, 0) # Remove self-loops
partial_corr

```

```
Out[12]: array([[ 0.          ,  0.05949518, -0.          ,  0.          ,  0.06795808,
                  0.          , -0.          ,  0.          ,  0.          ,  0.          ,
                  0.32167036,  0.15466485],
 [ 0.05949518,  0.          , -0.          ,  0.          ,  0.17875531,
                  0.          ,  0.          ,  0.          ,  0.          ,  0.          ,
                  0.16728484,  0.08265936],
 [-0.          , -0.          ,  0.          , -0.          , -0.          ,
                  -0.          , -0.04547375, -0.          , -0.          , -0.          ,
                  -0.          , -0.          ],
 [ 0.          ,  0.          , -0.          ,  0.          ,  0.          ,
                  0.06993187, -0.          ,  0.          ,  0.          ,  0.14693553,
                  0.          ,  0.0778853 ],
 [ 0.06795808,  0.17875531, -0.          ,  0.          ,  0.          ,
                  0.          , -0.          ,  0.          ,  0.          ,  0.          ,
                  0.17445913,  0.16837075],
 [ 0.          ,  0.          , -0.          ,  0.06993187,  0.          ,
                  0.          , -0.          ,  0.          ,  0.          ,  0.49867793,
                  0.          ,  0.08214683],
 [-0.          ,  0.          , -0.04547375, -0.          , -0.          ,
                  -0.          ,  0.          ,  0.          , -0.          , -0.          ,
                  -0.          , -0.          ],
 [ 0.          ,  0.          , -0.          ,  0.          ,  0.          ,
                  0.          ,  0.          ,  0.          ,  0.          ,  0.07034034,
                  0.          ,  0.03410157],
 [ 0.          ,  0.          , -0.          ,  0.          ,  0.          ,
                  0.          , -0.          ,  0.          ,  0.          ,  0.          ,
                  0.          ,  0.09821294],
 [ 0.          ,  0.          , -0.          ,  0.14693553,  0.          ,
                  0.49867793, -0.          ,  0.07034034,  0.          ,  0.          ,
                  0.03307653,  0.29081113],
 [ 0.32167036,  0.16728484, -0.          ,  0.          ,  0.17445913,
                  0.          , -0.          ,  0.          ,  0.          ,  0.03307653,
                  0.          ,  0.35639575],
 [ 0.15466485,  0.08265936, -0.          ,  0.0778853 ,  0.16837075,
                  0.08214683, -0.          ,  0.03410157,  0.09821294,  0.29081113,
                  0.35639575,  0.          ]])
```

Once we obtain the partial correlation matrix, we will use it to generate the adjacency matrix. We use the rule that if the absolute value of the partial correlation between to nodes are greater than 0, the entry for the adjacency matrix will 1. Otherwise the entry will be 0.

```
In [13]: # Create adjacency matrix using a threshold
threshold = 0.0 # Adjust as needed
adjacency = (np.abs(partial_corr) > threshold).astype(int)
adjacency
```

```
Out[13]: array([[0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1],
               [1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1],
               [0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
               [0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1],
               [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
               [0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1],
               [0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
               [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1],
               [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
               [0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1],
               [1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1],
               [1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0]])
```

Now, we are ready to draw the network!

```
In [14]: # Create network
G = nx.from_numpy_array(adjacency)

# Relabel nodes with asset and factor names
mapping = {i: name for i, name in enumerate(returns.columns)}
G = nx.relabel_nodes(G, mapping)

# Assign types to nodes
node_types = {
    'Apple': 'Asset',
    'Amazon': 'Asset',
    'USD': 'Factor',
    'GE': 'Asset',
    'Google': 'Asset',
    'JPMorgan': 'Asset',
    'Treasury': 'Factor',
    'Warner-Bro': 'Asset',
    'Walmart': 'Asset',
    'Financial': 'Factor',
    'Technology': 'Factor',
    'SP500': 'Factor',
}

nx.set_node_attributes(G, node_types, 'type')

# Draw the network graph
plt.figure(figsize=(14, 10))

# Use different colors for assets and factors
node_colors = ['skyblue' if G.nodes[name]['type'] == 'Asset' else 'orange' for name in G.nodes]

# Create layout
pos = nx.spring_layout(G, seed=42)

# Draw the network
nx.draw(G, pos,
        with_labels=True,
        node_size=500,
        node_color=node_colors,
        font_size=10,
```

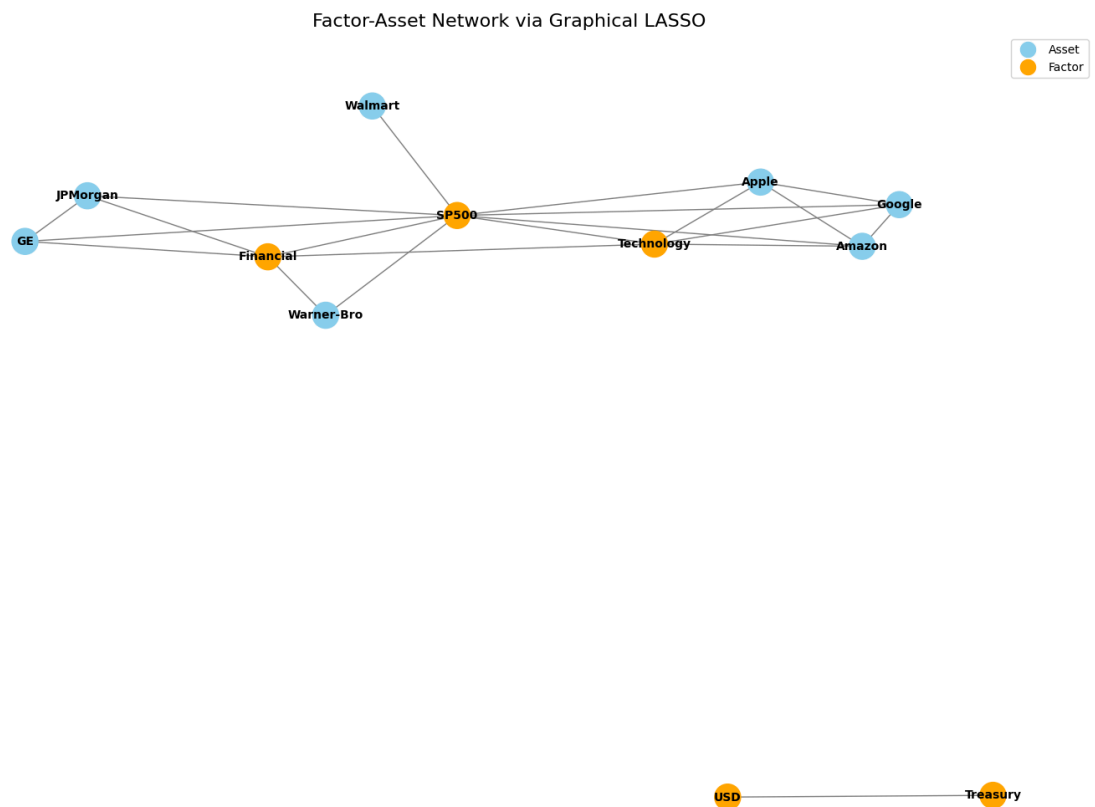
```

font_weight='bold',
edge_color='gray',
arrows=True,
arrowsize=15)

# Add a Legend
asset_patch = plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='skyblue',
                        markersize=15, label='Asset')
factor_patch = plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='orange',
                          markersize=15, label='Factor')
plt.legend(handles=[asset_patch, factor_patch], loc='upper right')

plt.title('Factor-Asset Network via Graphical LASSO', fontsize=16)
plt.show()

```



From the above asset and factor network graph, we have some interesting observations.

1. S&P500 ETF, Financial ETF and Technology ETF take up the central section of the network, representing their high connections with other stocks.
2. Treasury EFT and USD index only connect to each other and forms a separate cluster, demonstrating their lack of connection to the 7 stocks
3. Three technology stock returns (Apple, Amazon, and Google) form a cluster and are connected to each other. They are also connected to technology ETF and S&P500 index.
4. JP Morgan, GE and financial ETF all connect to each other and form a financial cluster.
5. Walmart is only connected to S&P500 index.

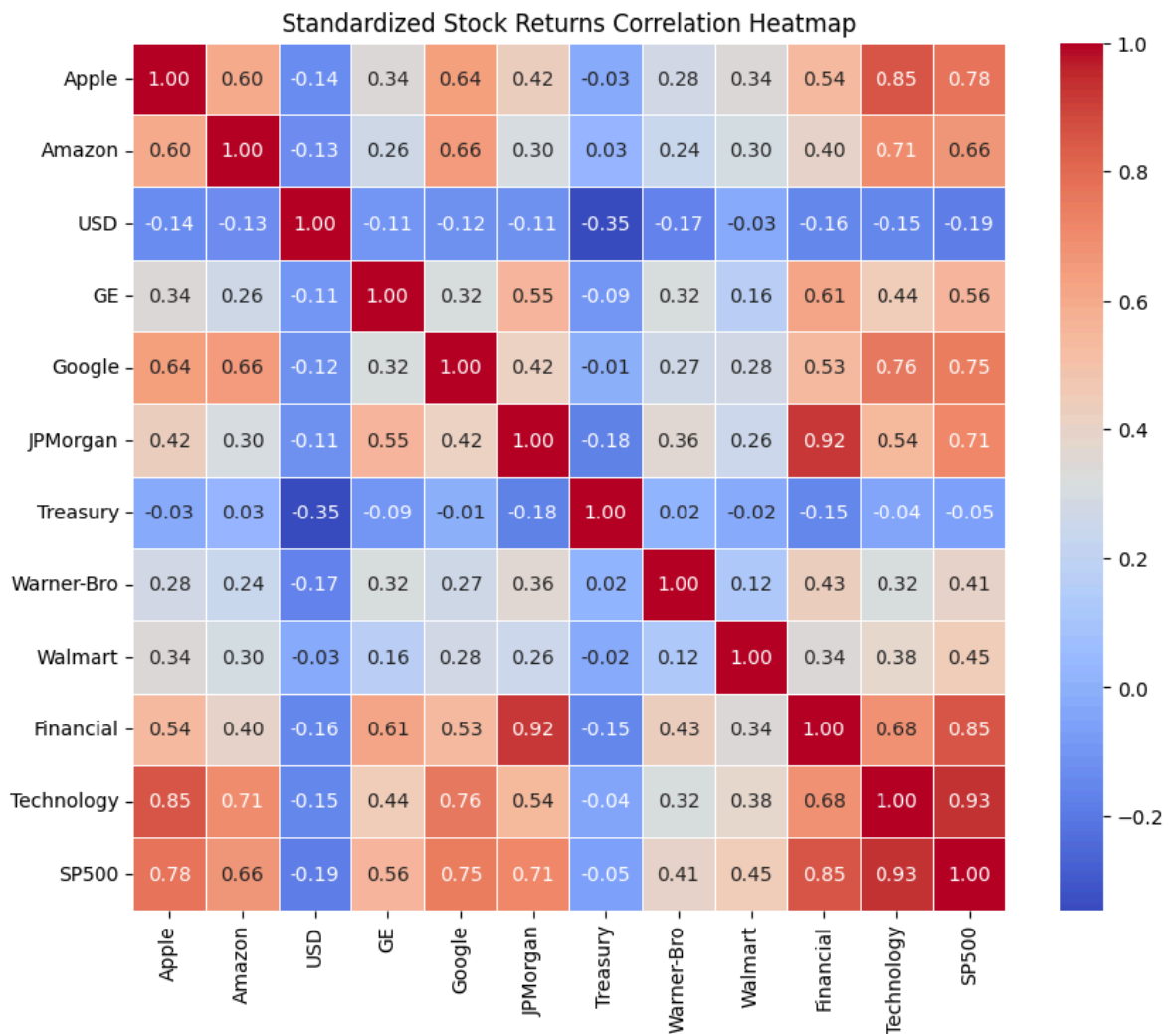
With the above observations, we can see the 3 technology stocks are correlated. It can be an issue if all 3 stocks have significant allocation in one portfolio. The portfolio manager may have to review the asset allocation and adjust the holdings of these 3 technology stocks. Another insight we can get from the network graph is that treasury ETF and USD are not correlated to the equity markets. They can be good alternative investment options to add to an all equity portfolio to diversity in equity market risk.

7.4 Network Comparison with Correlation Matrix and PCA

We analyzed the result of the asset factor network in the last section. While the Graphical LASSO-based network reveals conditional dependencies between assets and factors, it is interesting compare these findings with traditional correlation analysis and Principal Component Analysis (PCA). Let's start with the correlation analysis. We will create a correlation matrix and draw a heatmap for analysis.

```
In [15]: # Calculate the correlation matrix for standardized stock returns and draw a correlation heatmap
corr_matrix = standardized_returns.corr()

# Create the heatmap
plt.figure(figsize=(10, 8)) # Adjust figure size as needed
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Standardized Stock Returns Correlation Heatmap')
plt.show()
```



From the above correlation heatmap, we can see similar conclusions we obtained from the asset and factor network. What is interesting is the the asset and factor network provides more sparse connections in the Graphical LASSO network. In this heatmap, we see decent correlations between Apple and Financial ETF, JPMorgan and Technology ETF, and Google and Financial ETF pairs. However, we don't see the links in the asset factor network. This suggests that the additional high correlation pairs from the heatmap are mediated through other variables in the asset factor network.

Next, let's check PCA analysis.

```
In [16]: from sklearn.decomposition import PCA

pca = PCA(n_components=12)
pca.fit(standardized_returns)

# Get proportions of variance and cumulative proportion of variance
pr_var = pca.explained_variance_ratio_
cum_pr = np.cumsum(pca.explained_variance_ratio_)
ind = ["Proportion of variance", "Cumulative proportion of variance"]
```

```
cols = ["PC1", "PC2", "PC3", "PC4", "PC5", "PC6", "PC7", "PC8", "PC9", "PC10", "PC11"]
pd.DataFrame(np.vstack((pr_var, cum_pr)), ind, columns=cols)
```

Out[16]:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
Proportion of variance	0.473912	0.119811	0.104756	0.069253	0.060848	0.050367	0.041748	0.034372
Cumulative proportion of variance	0.473912	0.593723	0.698479	0.767732	0.828580	0.878948	0.920696	0.955067

From the above PCA result, we can see that the first component takes up 47% of data variation. The first 3 components account for almost 70% of the data variation. It means that there are concentrated factors for our dataset. Now let's check the loadings.

In [17]: *# Coefficients (Loadings) of 11 Principal Components*

```
pc_res = pd.DataFrame(pca.components_.T, index=list(standardized_returns.columns),
pc_res
```

Out[17]:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Apple	0.336618	0.156110	0.229826	-0.108929	-0.020074	-0.051825	0.016426
Amazon	0.293807	0.257706	0.302293	-0.199291	0.069379	0.012033	0.200793
USD	-0.086461	-0.477991	0.501285	-0.061011	0.255516	0.626710	-0.219379
GE	0.255220	-0.249909	-0.321528	-0.037276	-0.198923	0.439142	0.718808
Google	0.326271	0.171343	0.233276	-0.210983	-0.030126	0.009317	-0.104348
JPMorgan	0.315332	-0.323992	-0.285073	0.100327	-0.192931	-0.017013	-0.430174
Treasury	-0.030339	0.636881	-0.293956	0.155576	-0.007272	0.634414	-0.278162
Warner-Bro	0.197936	-0.030165	-0.374503	-0.205180	0.876363	-0.066143	0.015382
Walmart	0.191042	0.033586	0.214895	0.895470	0.247905	-0.062852	0.171800
Financial	0.361110	-0.249894	-0.238863	0.103423	-0.124025	-0.011632	-0.292428
Technology	0.384452	0.117194	0.186331	-0.095951	-0.062452	0.001550	-0.012022
SP500	0.408738	0.011034	0.028820	0.025596	-0.054673	0.015400	-0.076668

The above loading table gives us some key insights:

1. The key elements in PC1 are S&P500 ETF, Technology ETF and Financial ETF. It represents the overall equity market factor
2. The key elements in PC2 are USD index and Treasury ETF. This component captures the similar cluster of USD index and Treasury ETF cluster in the asset and factor network

3. PC3 seems to be the component that have negative impact on Financial sector but positive impact on technology sector.

Overall, the result from PAC is complement to the result from asset factor network. The network analysis tells us the connections and among assets and factors while PCA provides insights what are the possible common drivers that form these dependencies.

8. Conclusion

In this lesson, we first introduced what network theory is. We first discussed the basics of network theory. We then move on to demonstrate how regressions can incorporate into network theory to analyze the interconnections among stocks. We applied linear and non-linear models to the analysis. We also added a confounding variable to the linear regression model and explained the importance of a confounder. We ended the lesson with the introduction of graphical LASSO model. Graphical LASSO model can handle more variables in an efficient way. We then compare the result from Graphical LASSO with correlation matrix and PCA. With these comparison, we showed how these analytical methods can complement one another to provide different aspects of insights for the asset factor analysis.

References

1. Teo Paoletti (The College of New Jersey), "Leonard Euler's Solution to the Konigsberg Bridge Problem," Convergence (May 2011)
2. Konstantinov, Gueorgui S, and Fabozzi, Frank J. "Foundations for Building Portfolio Networks – Statistical and Econometric Models." Network Models in Finance, Wiley, 2025
3. López de Prado, M. (2023) Where are the factors in factor investing? Journal of Portfolio Management, 49(5), pp.6–20
4. Friedman J, Hastie T, Tibshirani R. Sparse inverse covariance estimation with the graphical lasso. Biostatistics. 2008 Jul;9(3):432-41. doi: 10.1093/biostatistics/kxm045. Epub 2007 Dec 12. PMID: 18079126; PMCID: PMC3019769.

FINANCIAL ECONOMETRICS

MODULE 7 | LESSON 3

GRANGER CAUSALITY AND NETWORK THEORY

Reading Time	60 minutes
Prior Knowledge	OLS, Covariance Matrix, VAR model, Basic Network Theory, Basic Python
Keywords	Association, Causality, Granger Causality

1. Introduction

In the last lesson, we talked a lot about one variable influences another variable in a network many times. In this lesson, we are going to go one step further and discuss various types of relationships among variables. We will first introduce the concept of association among variables and its characteristics. Next, we will give a short explanation about causality. Then we will introduce Granger causality and explain its assumptions and limitations. We will also provide some Granger causality application examples. Finally, we will show how to combine Granger causality with network to analyze relationship among time series.

2. Association Versus Causality

2.1. Understand Association

Association or **correlation** is used to describe the statistical relationship or pattern of two time series(Nicolae et al, 2022). The correlation coefficient of two time series tells us how strong and what direction of the association of the two time series.

Some important properties of the association are listed as follows:

1. The relationship is symmetric, which means if X is associated with Y, Y is also associated with X
2. Association does not indicate if one variable influences or has direct connection to the other variable. We also call it non-directional
3. Association can exist even two variables do not have direct causal connection

4. Association can be measured using correlation coefficients

If the correlation coefficient shows that two time series are highly associated (correlated), our next question is usually if one time series could cause the other time series. There can be two possible causal scenarios:

1. Direct Causation: change in variable X causes change in variable Y (or vice-versa)
2. Common response: variable X and variable Y are influenced by variable Z

We can use the network graph we learned in the last lesson to demonstrate the two scenarios.

Figure 1. Network Graph for Causal Types of Association



From figure 1, we can see Direct Causation situation on the left is the most straight forward causal relationship. Common Response situation on the right is the same as the complicated case we had with a confounding variable C in the last lesson. When a confounding variable is not included in a model, the model result will create a spurious association. Identifying and including a confounding variable is best practice to remove the spurious association between A and B.

2.2. Understand Causality

Causality means that one variable directly influences or makes changes in another (Hansen, 2014). Causal Inference is the method to analyze actual cause-and-effect relationship among variables.

A variable x_1 is considered to have a causal influence on the outcome variable y if changing x_1 leads to a change in y , assuming all other variables remain fixed. The complete model for the outcome can be expressed as: $y = h(x_1, x_2, u)$

where x_1 and x_2 are observable inputs,
 u is an unmeasured random component,
 h describes the functional relationship among these elements.

The causal effect of x_1 is defined as the rate of change in y with respect to x_1 , while keeping x_2 and u constant: $C(x_1, x_2, u) = \nabla_1 h(x_1, x_2, u)$

where ∇_1 is the partial derivative with respect to x_1 .

Here are the key properties of causality:

1. Causality represents asymmetric relationship, which means even X causes Y, it doesn't mean Y causes X
2. Causality shows which variable influences the other. It is also called directional
3. Causality is sequential, which means cause precedes effect

4. There is a mechanism to make influence happen

3. Granger Causality

3.1. Introduction

In the last section, we discussed association and causality. In this section, we are going to introduce another related method to analyze the relationship of two time series - Granger Causality. **Granger Causality** is a framework introduced by Granger(1969) to access if the past values of one time series can predict the other time series. For example, Granger causality is used to check if the historical values of x_t is predictive of y_t . If the answer is yes, we say x_t Granger causes y_t . Granger causality leverages the feature that there is an ordering of values in a time series to study cause-and-effect relationship for time series. One thing we need to be careful is even x_t Granger causes y_t does not mean x_t directly causes y_t . Granger causality is different from the causality we discussed in the last section. Granger causality focuses more on using the history of one variable to predict another variable and not actual cause-and-effect analysis.

Researchers have been using Granger causality to find out interactions among variables in finance and macroeconomics. For example, Billio et al (2012) used Granger causality to understand the connectivity and systemic risk of financial institutions. However, there are limitations about Granger causality to provide causal information among time series. We will discuss these limitations in the later part of this section. First, let's give a formal description of Granger Causality.

3.2. Concept of Granger Causality

We have two **stationary** time series x_t and y_t . With these two time series, we can define the following two information sets: $\mathcal{H}_{1,t} = (y_t, y_{t-1}, y_{t-2}, \dots)$ and $\mathcal{H}_{2,t} = (y_t, x_t, y_{t-1}, x_{t-1}, y_{t-2}, x_{t-2}, \dots)$ where information set $\mathcal{H}_{1,t}$ includes information from y_t only and information set $\mathcal{H}_{2,t}$ includes information from both y_t and x_t .

We say x_t 'Granger causes' y_t if

$$\text{var}[y_t - \mathcal{P}(y_t | \mathcal{H}_{2,t-1})] < \text{var}[y_t - \mathcal{P}(y_t | \mathcal{H}_{1,t-1})] \quad (1)$$

where $\mathcal{P}(y_t | \mathcal{H}_{2,t-1})$ is the optimal prediction of y_t , given the information set $\mathcal{H}_{2,t-1}$ and $\mathcal{P}(y_t | \mathcal{H}_{1,t-1})$ is the optimal prediction of y_t , given the information set $\mathcal{H}_{1,t-1}$

Equation (1) says when the variance of prediction errors of y_t given information set $\mathcal{H}_{2,t-1}$ is smaller than the variance of prediction errors of y_t given information set $\mathcal{H}_{1,t-1}$, the time series x_t Granger causes y_t . It means the addition of past values of x_t in the model improves the prediction of y_t and reduces the variance of prediction errors.

Based on the above framework, how can we implement the actual Granger causality test?

We can start by writing down the following two OLS regression models:

(reduced model) $y_t = \alpha_0 + \alpha_1 y_{t-1} + \alpha_2 y_{t-2} \dots + \alpha_p y_{t-p} + \epsilon_{t(H_1)}$

(full model) $y_t = \alpha_0 + \alpha_1 y_{t-1} + \alpha_2 y_{t-2} \dots + \alpha_p y_{t-p} + \beta_1 x_{t-1} + \beta_2 x_{t-2} \dots + \beta_p x_{t-p} + \epsilon_{t(H_2)}$

where α_i are coefficients of y_i and β_j are coefficients of x_j while $i, j \in [1, 2, \dots, p]$

$\epsilon_{t(H_1)}$ is the white noise for the reduced model

$\epsilon_{t(H_2)}$ is the white noise for the full model

Equation (2) represents the model that only uses the past values of y to predict y_t

while equation (3) represents the model that uses not only the past values of y but also the past values of x to predict y_t . Equation (2) uses information set

H_1 hence this model is a reduced model only using partial information.

Equation (3) uses information set H_2 therefore this model is a full model fusing full information. To check if x Granger causal to (or Granger causes) y , we can write down the following hypothesis testing:

$H_0: \beta_1 = \beta_2 = \dots = \beta_p = 0$

$H_1: \text{at least one of the } \beta_s \neq 0$

To test the above hypothesis, we will use F -test. Our test statistic is as follows:

$F = \frac{(RSS_{\text{reduced}} - RSS_{\text{full}}) / (r - s)}{(RSS_{\text{full}}) / (T - r)}$

where RSS is residual sum of squares of the regression model

r is the number of parameters in the full model

s is the number of parameters in the reduced model

T is the number of samples

If the p value of the F -test is less than 5% given the significance level is set at 5%, we can reject the null hypothesis (H_0) and say x Granger causes y . We will then switch the role of x and y to repeat the above said procedure to check if y Granger causes x .

3.3. Vector Autoregression Model (VAR) and Granger Causality

In the last section, we described the detailed process of how to test if x Granger causes y . And then use the same process to test if y Granger causes x . Many times, we can present the last two Granger causality tests in a bivariate VAR(p) framework. Let's first write down a VAR model for x and y with lag of p .

$$\begin{aligned}
 x_t &= \phi_0 + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \dots + \phi_p x_{t-p} + \theta_1 y_{t-1} + \theta_2 y_{t-2} + \dots + \theta_p y_{t-p} + \varepsilon_t \\
 y_t &= \alpha_0 + \alpha_1 y_{t-1} + \alpha_2 y_{t-2} + \dots + \alpha_p y_{t-p} + \beta_1 x_{t-1} + \beta_2 x_{t-2} + \dots + \beta_p x_{t-p} + \varepsilon_t
 \end{aligned}$$

Then we estimate this VAR(p) model. Once we have the coefficient estimates of the model, we can conduct Granger causality test. For example, if we want to test whether x Granger causes y , it is equal to test $H_0: \beta_1 = \beta_2 = \dots = \beta_p = 0$. If we want to test whether y Granger causes x , it is equal to test $H_0: \theta_1 = \theta_2 = \dots = \theta_p = 0$. Another reason we will use VAR(p) framework for Granger causality test is the selection of the number of lags p . That leads to our next topic: the assumptions of Granger causality.

3.4. Assumptions and Limitations of Granger Causality

Now you have seen how to set up a test for Granger causality for two time series. You might also have seen a few assumptions you used during the description of the process. Let's summarize the key assumptions here.

Stationarity

The two time series need to be stationary before applying Granger causality test. We can use augmented Dickey-Fuller test to check for stationarity for the time series. If any time series is not stationary, we can apply the methods we learned from previous lessons to transform the time series.

Linearity

Granger causality test assumes the time series are in a linear relationship.

Continuous-Valued Time Series

The values of the time series for Granger causality test need to be continuous.

No Confounding Variables

All relevant variables are observed and included in the analysis and there are no unmeasurable variables.

Lag is known

The lag for the dependent past observations is known and uniform across all variables.

The linear dependency of the time series assumption can be restrictive as many time series data have non-linear relationship. The no confounding variables assumption is unrealistic especially for the bivariate model we having been discussing. We talked about the problem of non including confounding variables in the model in the last lesson. We will demonstrate this issue when we show the Granger causality and network combination example. As for known lag assumption, we can use the lag order selection function in the VAR model to decide the optimal lag for Granger causality test.

3.5. An Example for Granger Causality Test in Bivariate VAR model

In this section, let's try to run Granger causality for just two time series. We will pick returns of JPMorgan stock and Technology ETF for this example. As discussed in the last section, we need to check for stationarity of these two time series before conducting Granger causality test. We will use augmented Dickey Fuller test to check for stationarity.

```
In [1]: pip install networkx seaborn --upgrade
```

Requirement already satisfied: networkx in /usr/local/lib/python3.11/site-packages (3.3)

WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce4861e8950>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/networkx/

WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce4861e9710>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/networkx/

WARNING: Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce4861ea310>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/networkx/

WARNING: Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce4861eafd0>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/networkx/

WARNING: Retrying (Retry(total=0, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce4861ebb0>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/networkx/

Requirement already satisfied: seaborn in /usr/local/lib/python3.11/site-packages (0.13.2)

WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce48623bb10>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce48623ca90>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

WARNING: Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce48623d250>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

WARNING: Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce48623de10>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

WARNING: Retrying (Retry(total=0, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError('<pip._vendor.urllib3.connection.HTTPSConnection object at 0x7ce48623e910>: Failed to establish a new connection: [Errno -3] Temporary failure in name resolution')': /simple/seaborn/

Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.11/site-packages (from seaborn) (2.1.3)

Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.11/site-packages (from seaborn) (2.2.3)

Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.11/site-packages (from seaborn) (3.10.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.56.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/site-packages (from pandas>=1.2->seaborn) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/site-packages (from pandas>=1.2->seaborn) (2025.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
Note: you may need to restart the kernel to use updated packages.

```
In [2]: import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import yfinance as yf
import warnings
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.api import VAR

warnings.filterwarnings("ignore")
```

```
In [3]: # Gather stock data and factor data for 7 years

# List of assets (stocks)
assets = ['AAPL',    # Apple
          'WMT',     # Walmart
          'AMZN',    # Amazon
          'GOOGL',   # Google
          'JPM',     # JP Morgan
          'WBD',     # Warner Brothers
          'GE']      # General Electric Co.

# List of factors
factors = [
    '^GSPC',    # S&P 500 (market factor)
    'XLF',      # Financial Select Sector SPDR Fund
    'XLK',      # Technology Select Sector SPDR Fund
    'SHY',      # Short-term Treasury ETF (interest rate factor)
    'DX-Y.NYB'  # US Dollar index
]
```

```

# Combine assets and factors
tickers = assets + factors

# Download data
data_0 = pd.read_csv('M7L2_stock_prices.csv')

data = data_0.drop('Date', axis=1)

# Calculate returns
returns = data.pct_change().dropna()

# Add meaningful labels
returns.columns = [
    'Apple', 'Amazon', 'USD', 'GE', 'Google', 'JPMorgan', 'Treasury', 'Warner-Bro',
]

```

```

In [4]: # Standardize data
standardized_returns = (returns - returns.mean()) / returns.std()

```

```

In [5]: # Perform the ADF test for JPMorgan stock return
jp_result = adfuller(returns['JPMorgan'])

print('p-value of JPMorgan: %f' % jp_result[1])

```

p-value of JPMorgan: 0.000000

```

In [6]: # Perform the ADF test for Technology ETF
tech_result = adfuller(returns['Technology'])

print('p-value of Technology: %f' % tech_result[1])

```

p-value of Technology: 0.000000

From the results of the above augmented Dickey Fuller tests, both time series are stationary. Next we will use select lag function from VAR model to find the optimal lag order for Granger causality test.

```

In [7]: # Use optimal lag order to run Granger Causality test
from statsmodels.tsa.api import VAR

# Create a dataset to run the VAR model
gc_data = returns[["JPMorgan", "Technology"]]

# Use select_order to find the optimal lag order
model = VAR(gc_data)
lag_order_results = model.select_order(maxlags=10)
lag_order = lag_order_results.aic # You can also use bic, fpe or hqic
print('Optimal Lag Order:', lag_order)

```

Optimal Lag Order: 9

The best lag order is 9 based on the above code result. With that information, now we can run the VAR model with the best lag order and run Granger causality test.

```
In [8]: # Fit the VAR model
model_fitted = model.fit(lag_order)
```

With the fitted model, we can now to test Granger causality for time series 'Technology' and time series 'JPMorgan'. We will test both directions.

```
In [9]: # Perform Granger causality test on JPMorgan Granger causes Technology; the test st
test_result = model_fitted.test_causality(caused='Technology', causing='JPMorgan',k
print(test_result.summary())
```

Granger causality F-test. H₀: JPMorgan does not Granger-cause Technology. Conclusion: reject H₀ at 5% significance level.

```
=====
Test statistic Critical value p-value          df
-----
                2.353          1.883  0.012 (9, np.int64(3462))
-----
```

```
In [10]: # Perform Granger causality test on Technology Granger causes JPMorgan; the test st
test_result = model_fitted.test_causality(caused='JPMorgan', causing='Technology',k
print(test_result.summary())
```

Granger causality F-test. H₀: Technology does not Granger-cause JPMorgan. Conclusion: reject H₀ at 5% significance level.

```
=====
Test statistic Critical value p-value          df
-----
                3.105          1.883  0.001 (9, np.int64(3462))
-----
```

From the above Granger causality test results, we can see that JPMorgan and Technology ETF Granger cause each other. These two time series have dynamic interactions with each other.

4. Granger Causality and Network Theory

In this section, we are going to combine Granger causality with network theory to study interconnection of multiple time series. We will have stock and factor returns as nodes. Their link will be determined by Granger causality. If asset \$X\$ Granger causes asset \$Y\$, there will be an arrow pointing to asset \$Y\$ from asset \$X\$. Then we can draw the network graph to understand the interconnections among assets and factors based on Granger causality. Let's begin by checking the stationarity for all time series in the dataset.

```
In [11]: # Check for Stationarity for all time series
results = {'Column': [], 'P-Value': [], 'Stationary (5% level)': []}

# Loop through each column
for column in returns.columns:
    adf_result = adfuller(returns[column])

    # Extract p-value (second value in the result tuple)
    p_value = adf_result[1]
```

```

# Add results to dictionary
results['Column'].append(column)
results['P-Value'].append(p_value)
results['Stationary (5% level)'].append('Yes' if p_value < 0.05 else 'No')

results_df = pd.DataFrame(results)
print(results_df)

```

	Column	P-Value	Stationary (5% level)
0	Apple	9.268138e-25	Yes
1	Amazon	0.000000e+00	Yes
2	USD	2.046682e-30	Yes
3	GE	2.764974e-23	Yes
4	Google	3.329989e-26	Yes
5	JPMorgan	1.046926e-22	Yes
6	Treasury	2.080702e-14	Yes
7	Warner-Bro	0.000000e+00	Yes
8	Walmart	1.088815e-26	Yes
9	Financial	3.848073e-24	Yes
10	Technology	1.931759e-25	Yes
11	SP500	3.877946e-24	Yes

From the above result, we can see all asset and factor returns are stationary. Now we can move to the next step. We will follow the Granger causality test process for bivariate VAR model example in the last section and loop through all the pairs in the dataset. If the Granger causality test is statistically significant for a pair of time series, the entry of the adjacency matrix of this pair will be 1. Otherwise, the entry will be 0. We also apply Bonferroni method to adjust P value for statistical level. Finally, we will use the filled adjacency matrix to draw the network graph.

```

In [12]: # Run a pairwise VAR model and then Granger Causality Asset and Factor Network
n_var = len(returns.columns)
adj_matrix = np.zeros((n_var, n_var))

# Significance threshold (adjusted based on Bonferroni Method)
p_threshold = 0.05/(n_var*(n_var-1))

# Loop through all pairs of assets
for i, return_i in enumerate(returns):
    for j, return_j in enumerate(returns):
        if i != j: # Skip self-relationships
            var_data = returns[[return_i, return_j]]
            # Use select_order to find the optimal lag order
            model = VAR(var_data)
            lag_order_results = model.select_order(maxlags=10)
            lag_order = lag_order_results.aic # You can also use bic, fpe or hqic
            if lag_order > 0:
                final_order = lag_order
            else:
                final_order = 1
            # Fit the VAR model
            model_fitted = model.fit(final_order)
            # Run conditional Granger Causality
            test_result = model_fitted.test_causality(caused= return_j, causing= re

```

```

        # Get p-value for the Granger causality test
        p_value = test_result.pvalue

        # If significant, add connection to adjacency matrix
        adj_matrix[i, j] = 1 if p_value < p_threshold else 0

# Create directed graph from adjacency matrix
G = nx.from_numpy_array(adj_matrix, create_using=nx.DiGraph)

# Relabel nodes with asset names
mapping = {i: name for i, name in enumerate(returns)}
G = nx.relabel_nodes(G, mapping)

```

```

In [13]: # Create and visualize the network

# Assign types to nodes
node_types = {
    'Apple': 'Asset',
    'Amazon': 'Asset',
    'USD': 'Factor',
    'GE': 'Asset',
    'Google': 'Asset',
    'JPMorgan': 'Asset',
    'Treasury': 'Factor',
    'Warner-Bro': 'Asset',
    'Walmart': 'Asset',
    'Financial': 'Factor',
    'Technology': 'Factor',
    'SP500': 'Factor',
}

nx.set_node_attributes(G, node_types, 'type')

# Draw the network graph
plt.figure(figsize=(14, 10))

# Use different colors for assets and factors
node_colors = ['skyblue' if G.nodes[name]['type'] == 'Asset' else 'orange' for name

# Create Layout
pos = nx.spring_layout(G, seed=42)

# Draw the network
nx.draw(G, pos,
        with_labels=True,
        node_size=500,
        node_color=node_colors,
        font_size=10,
        font_weight='bold',
        edge_color='gray',
        arrows=True,
        arrowsize=15)

# Add a Legend
asset_patch = plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='skyblue'

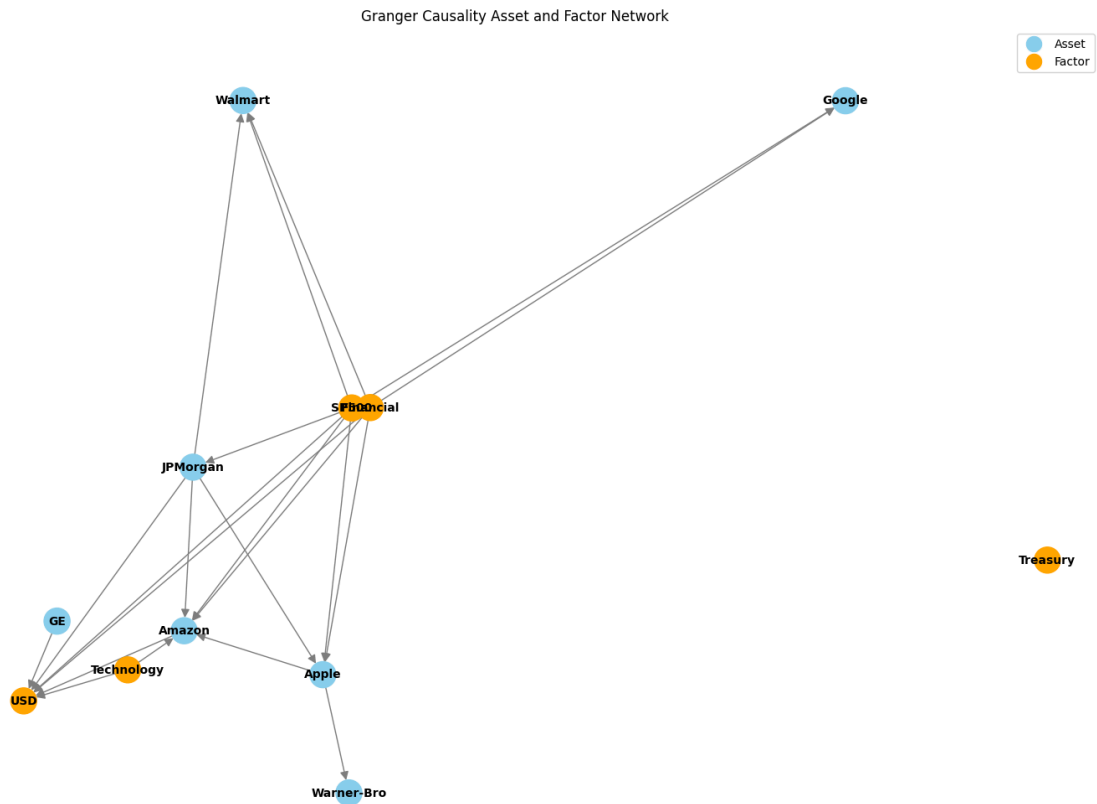
```

```

        markersize=15, label='Asset')
factor_patch = plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='orange',
        markersize=15, label='Factor')
plt.legend(handles=[asset_patch, factor_patch], loc='upper right')

plt.title('Granger Causality Asset and Factor Network')
plt.show()

```



The above network graph provides some interesting Granger causality test results. From the graph, we can see SP500 and Financial ETFs Granger cause all the stock returns except GE and Warner Brothers. One interesting observation is that several stocks also Granger cause USD ETF. For example, not only SP500 Granger causes USD ETF, Amazon also Granger causes USD ETF. There might be a spurious relationship going on here. As we discuss before Granger causality test in bivariate VAR model setting can have no confounding variable issue. In the above graph, we see SP500 Granger causes USD ETF. SP500 Granger also Granger causes Amazon. Then Amazon also Granger causes USD ETF. SP500 might be the confounding variable when we conduct Granger causality test between Amazon and USD ETF. Let's check on that by running the following 3 variable VAR model. As before, we will find the optimal lag order and fit the model. Once the model is fitted, we can conduct Granger causality test.

```

In [14]: # Create a dataset to run the VAR model
gc_data = returns[["Amazon", "SP500", "USD"]]

# Use select_order to find the optimal lag order

```

```

model = VAR(gc_data)
lag_order_results = model.select_order(maxlags=10)
lag_order = lag_order_results.aic # You can also use bic, fpe or hqic
# Fit the VAR model
model_fitted = model.fit(lag_order)

```

```

In [15]: # Perform Granger causality test on JPMorgan Granger causes Technology; the test st
test_result = model_fitted.test_causality(caused='USD', causing='Amazon', kind='f')
print(test_result.summary())

```

Granger causality F-test. H₀: Amazon does not Granger-cause USD. Conclusion: fail to reject H₀ at 5% significance level.

```

=====
Test statistic Critical value p-value          df
-----
                1.673          1.882  0.090 (9, np.int64(5166))
-----

```

From the above Granger causality test result, we see that P value is 0.09 and we cannot reject H_0 and the result is Amazon does not Granger cause USD ETF. This is the example how omitting a confounding variable can change the result of Granger causality test. In the recent research advance, searchers have proposed several modifications to handle no confounding variable issue. We will not cover them in this lesson. For those who are interested, there is plenty information online.

5. Conclusion

In this lesson, we learned the differences between association and causality. Then we introduced Granger causality and went through its theory. We also discussed the assumptions and limitations of Granger causality. Finally we combined Granger causality and network theory to produce a Granger causality based network. This complex network can help us to better understand the Granger causal relationship among assets and factors.

References

1. Nicolae et al, "Part I: Exploring Data, 10. Data Collection, Causality and Association", Introduction to Data Science, University of Chicago, 2022, <https://ds1.datascience.uchicago.edu/10/data-collection.html>
2. Hansen "Conditional Expectation and Projection: Causal Effect ", Econometrics, University of Wisconsin, 2014, pp. 43
3. Billio, M., Getmansky, M., Lo, A. W. and L. Pelizzon. (2012) Econometric measures of connectedness and systemic risk in the finance and insurance sectors. Journal of Financial Economics, 104(3), pp. 535–559.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. PennState, Eberly College of Science. "10: Discriminant Analysis." STAT 505: Applied Multivariate Statistical Analysis. Lecture. <https://online.stat.psu.edu/stat505/lesson/10>

- **Required pages:** Sections 10.1, 10.2, 10.3, and 10.4
- **Estimated reading time:** 1 hour 30 minutes

FINANCIAL ECONOMETRICS

MODULE 7 | LESSON 4

SUPERVISED LEARNING AND UNSUPERVISED LEARNING MODELS

Reading Time 90 minutes

Prior Knowledge Linear Algebra, Basic statistics, Regression Analysis

Keywords Supervised learning, Unsupervised learning, Classification, Clustering, Linear discriminant analysis, Hierarchical clustering, K-Means procedure

In this lesson, we are going to explore basic concepts in machine learning. We will have a new module dedicated to the topic of machine learning, but in this lesson we are going to provide a high-level overview of machine learning and introduce several basic concepts in this field. We will leave the in-depth discussion of various machine learning topics for the next module. We will start by introducing the concepts of supervised learning and unsupervised learning. Then we will present one model from the supervised learning group and one model from the unsupervised learning group. The model from supervised learning group will be about classification and, specifically, it will rely on discriminant analysis. The model from the unsupervised group, on the other hand, will be based on cluster analysis. We will delve into a deeper discussion about discriminant analysis while for cluster analysis we will limit ourselves to a brief overview. Cluster analysis will receive a deeper coverage in the Machine Learning class.

1. Supervised Learning

Supervised Learning means that a model is constructed to predict data with label. What does it mean in plain English? For example, if there 10 cats, 10 dogs and 10 chickens in the assigned dataset, for these 30 animals, we also have a description of their feature variables, like the shape of their noses, the shape of their mouth, fur type, body type etc... We would like to build a model using feature variables as input variables to predict if an animal is a cat, a dog or a chicken. Once the model is created, we are able to feed the features of a certain animal to this model and the model tells us what this animal is. This simple modeling concept is the same concept of a more complicated facial recognition model used in Apple's iPhone.

Labels data can be numeric or categorical. Our last animal example is a categorical example. One example for a supervised learning model with numeric labels of data is used to predict someone's income level. Assume we have 100 people with their income data, education level and other variables. In this case, to each person is attached a label that contains his or her income level. We would like to build a model based on certain information we have to predict income level. Does this model sound familiar? This is a regression model. Regression models that we have considered so far are part of the supervised learning in the field machine learning.

With the above examples, we can see that there are two types of supervised learning models: **Classification and Regression**.

Classification models: models for data with categorical label or with categorical output variable. For example, types of animals, positive or negative for a specific form of cancer, identify stocks as value or growth stocks, etc... Popular classification models include discriminant analysis, logistic regression, K nearest neighbors and naïve Bayes classifiers, decision tree and support vector machine.

Regression models: models for data with numerical label or with numerical output variable. For example, income level, stock return, GDP growth etc... Linear regression is the most popular one in this category.

2. Unsupervised Learning

Sometimes, for a data project, we may not have a target variable or an output variable that we want to predict. Instead, we would like to analyze the data and see if we can group the data according to a particular pattern, similarity or features etc... We can also say that with unlabeled data, we would like to use the information we have from the data to assign it to certain groups. Models that aim at grouping unlabeled data are defined as **Unsupervised Learning**. These models need to find the hidden pattern within the data in order to separate the data into group elements.

For instance, we can collect trading information from traders in the stock market. For example, we could collect information like types of stock traded, position holding duration, strategies used etc... With all this information, we can use an unsupervised learning model to identify traders with similar behaviors and group them together. After the grouping is done, we can investigate traders' behavior variables within the same group and decide what are the common traits inside that particular group. After investigating behavior variables, we may come up with certain particular groups like buy-and-hold traders, high frequency traders or technical traders etc...

Popular unsupervised learning models include hierarchical clustering, k-means clustering, and principal component analysis.

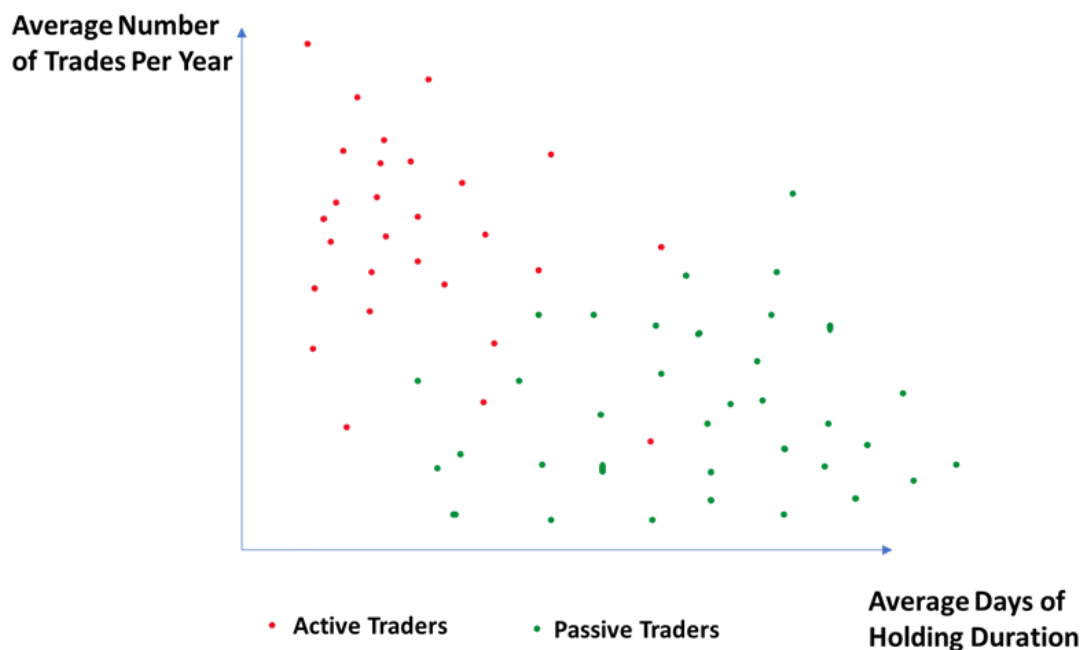
Supervised learning models and unsupervised learning models are two major modeling tools in machine learning. Under each modeling group, one can identify several modeling methods. In the following sections, we will introduce discriminant analysis from the supervised learning modeling group and clustering analysis from the unsupervised learning modeling group.

3. Discriminant Analysis

Discriminant analysis (DA) is a type of model used for supervised learning modeling. DA is used to predict the category of the subject given certain input data about the subject. One example we talked about in last section is to predict if an animal is a cat, a dog or a chicken given this animal's feature information. Hence, we also call DA a **Classification** model. A DA model is only used for analysis with categorical output.

A very simple example is beneficial to explain how DA works. Assume we have 70 active traders and 70 passive traders. We also have the data of their position holding duration in days and average number of trades they do per year. Our goal is to see if a trader's position holding duration and number of trades per year can tell us what type of trader he or she belongs to.

Figure 1. Scatter Plot of Active Traders and Passive Traders



In Figure 1, we can see that active traders usually have higher number of trades but short during for position holding and passive traders do the reverse. However, how can we use

these two input variables to better predict the type of a trader? We can use DA in this case. Specifically, we will use **Linear Discriminant Analysis (LDA)**.

3.1. Linear Discriminant Analysis (LDA): Theory

Linear Discriminant Analysis (LDA) provides a method to assign or classify a subject whose type is unknown to a type based on particular input data for this subject. In our example above, if we have a trader that we don't know whether he/she is an active or a passive trader, but we have the data regarding this trader's holding duration and number of trades, we can use LDA to predict if this trader is an active trader or a passive trader.

Assume that our dataset consists of 100 information about 100 individuals. In the dataset we have a categorical output variable K and 2 numerical input variables X and Y . Each data point contains information for K , X and Y . The output variable K contains 3 different types. Let's just call them type-a, type-b and type-c. LDA will create a model system as follows:

- When X_a and Y_a is from type a, $P(K_a|X_a, Y_a) > P(K_b|X_b, Y_b)$ and $P(K_c|X_c, Y_c)$
- When X_b and Y_b is from type b, $P(K_b|X_b, Y_b) > P(K_a|X_a, Y_a)$ and $P(K_c|X_c, Y_c)$
- When X_c and Y_c is from type c, $P(K_c|X_c, Y_c) > P(K_a|X_a, Y_a)$ and $P(K_b|X_b, Y_b)$

LDA supplies a model which creates a system to tell us the conditional probability of the data point being of each type given the input data. Furthermore, the model assigns the type with the highest conditional probability as the type from which input data comes from. From our above system, X and Y are assigned to type-a group when the conditional probability of being type-a is the highest compared to the conditional probability of being type-b or being type-c.

How do we get the conditional probability for each type? We will use Bayes' theorem and Bayesian statistics. For the conditional probability $P(K_a|X_a, Y_a)$, we can rewrite it as follows:

$$P(K_a | X_a, Y_a) = \frac{f(X_a, Y_a | K_a) P(K_a)}{\sum_{i=a,b,c} f(X_i, Y_i | K_i) P(K_i)}$$

In Bayesian statistics,

- $P(K_a|X_a, Y_a)$ is known as the posterior probability
- $f(X_a, Y_a | K_a)$ is called the likelihood function and
- $P(K_a)$ is the prior probability.

Since $\sum_{i=a,b,c} P(K_i)$ is the same for each conditional probability, we can drop it for the ease of calculation. Hence, we need to estimate $f(X, Y | K)P(K)$ for each type.

To estimate $P(K)$, we will calculate the ratio of number of data points of certain type over number of all data points from the dataset. The sum of $P(K)$ of all types should be 1\$.

For estimation of $f(X,Y|K)$, we need to have sample mean and variance-covariance matrix for each type from the dataset. One key point here is variance-covariance matrices from different types should all be the same for LDA. If variance-covariance matrices are different among types, then we need to use another model. We can use the dataset we get to calculate sample means and sample variance-covariance matrices for 3 types.

Figure 2. Scatter Plot for Input Variables X and Y and Output Variable K with 3 Types

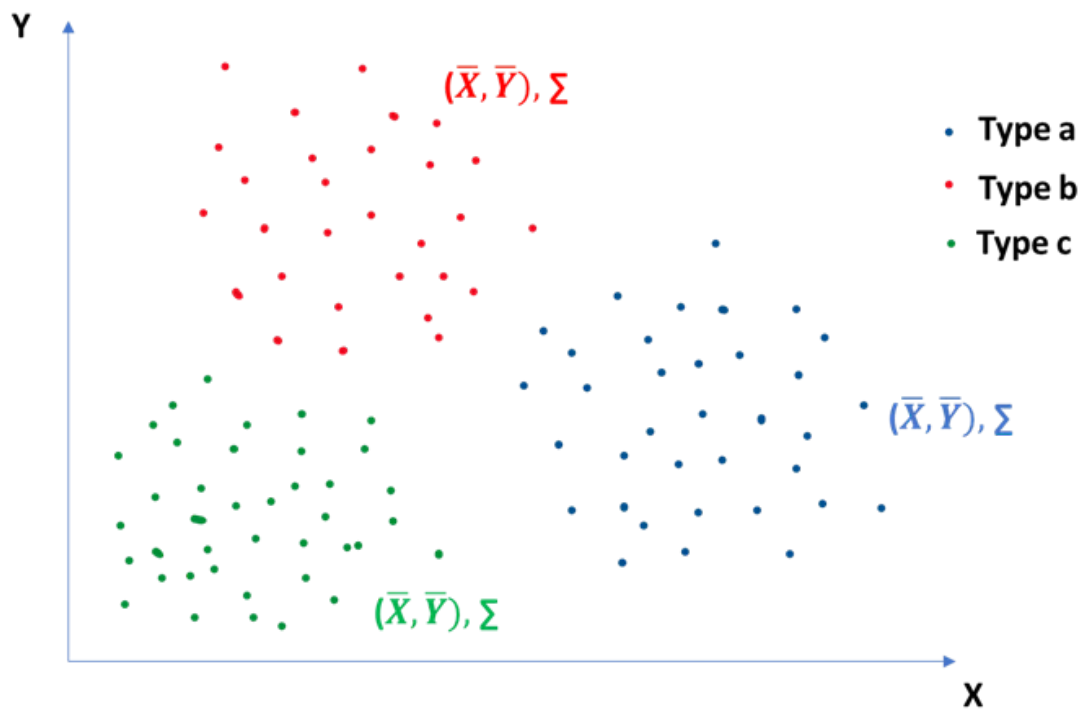


Figure 2 illustrates what we just discussed about. $\bar{X}, \bar{Y}$ are the sample means in each type. Σ is the variance-covariance matrix in each type. We require the following assumptions so we can apply LDA.

- a. $X|type$ and $Y|type$ are normally distributed
- b. Variance-Covariance matrices for different types are the same
- c. Each data point is independent of one another

With the above assumptions and the values of sample means and sample variance-covariance matrices, we can estimate the likelihood functions. With estimated likelihood function and prior probability, we can obtain an estimated posterior conditional probability for each type given the input data. In LDA, however, we use the above concept and we can derive 3 Linear Discriminant functions that are related to the 3 posterior conditional probability density functions. The 3 linear discriminant functions are shown as follows:

$$\begin{aligned} L_a &= \beta_{a,0} + \beta_{a,1} X + \beta_{a,2} Y \\ L_b &= \beta_{b,0} + \beta_{b,1} X + \beta_{b,2} Y \\ L_c &= \beta_{c,0} + \beta_{c,1} X + \beta_{c,2} Y \end{aligned}$$

Where

$$\begin{pmatrix} \beta_{a,0} & \beta_{a,1} & \beta_{a,2} \\ \beta_{b,0} & \beta_{b,1} & \beta_{b,2} \\ \beta_{c,0} & \beta_{c,1} & \beta_{c,2} \end{pmatrix}$$

is derived from sample means and sample variance-covariance matrices

The way to use linear discriminant functions is similar to use 3 posterior probability densities. If you have an untyped subject with data X and Y , you plug in X and Y in all 3 linear discriminant functions. The linear discriminant function that gives the highest output number indicates subject is from the type of the **linear discriminant function**. With linear discriminant functions, we can also derive posterior probabilities.

This is a very brief introduction to linear discriminant function. All the detail information about LDA will be in the required reading for this lesson. LDA gives us an idea of how classification can be done with a supervised model. There are other classification models that we will discuss in Machine Learning module.

3.2. Linear Discriminant Analysis (LDA): An Application

In this section, we provide a simple example of an application of LDA. Let's continue with our trader type example introduced earlier.

Assume we have the following linear discriminant functions for active traders and passive traders:

$$L_{\text{active}} = 132 - 1.42 \times \text{Holding Time} + 1.35 \times \text{Number of Trades}$$

$$L_{\text{passive}} = 211 + 0.52 \times \text{Holding Time} - 2.02 \times \text{Number of Trades}$$

Now we have a new trader and we would like to know this trader's type. This trader's holding time is 25 days and number of trades is 32 trades. Let's plug these numbers in to the two linear discriminant functions.

$$L_{\text{active}} = 132 - 1.42 \times 25 + 1.35 \times 32 = 139.7$$

$$L_{\text{passive}} = 211 + 0.52 \times 25 - 2.02 \times 32 = 90.36$$

From the above linear discriminant functions, this trader is an active trader because $L_{\text{active}} > L_{\text{passive}}$.

4. Cluster Analysis

In this section, we will talk about a popular method in unsupervised learning modeling group: **Cluster Analysis**. Unlike classification analysis, there is no categorical output variable in the dataset for cluster analysis. Cluster analysis analyze unlabeled dataset to find similarity among data points and create clusters from the data accordingly. One example is that you have 100 stocks and for each stock we know: company name, type of company, annual sales, margin, historical price, PE ratios etc... With all this information, we would like to segment the 100 stocks into clusters of similar characteristics. We don't have any preset view on these 100 stocks and we just want to use the available data to see if the data can offer insights on similarity in these stocks.

There are two methods to approach cluster analysis: **Hierarchical Clustering** and **K-Means Procedure**.

4.1. Hierarchical Clustering

Hierarchical Clustering begins with treating each data point as a cluster. It then uses a **linkage function** to decide which two data points are close to each other and combine them as a new cluster. Hierarchical clustering repeats the process until all data points are in one cluster. During this merge process, we can draw the following hierarchical clustering dendrogram.

Figure 3. Dendrogram Example

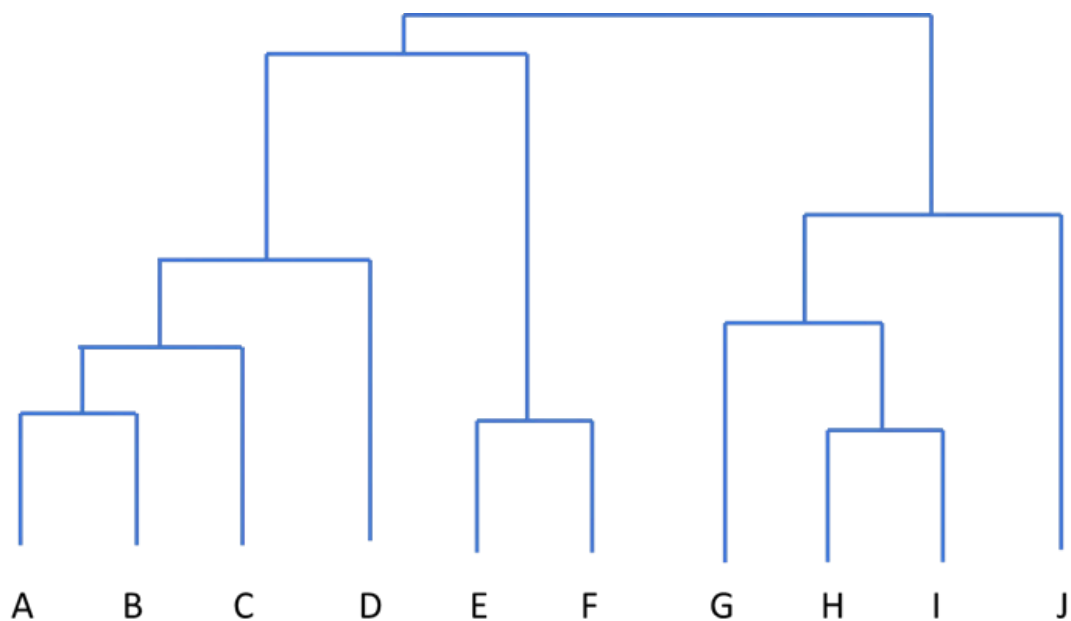
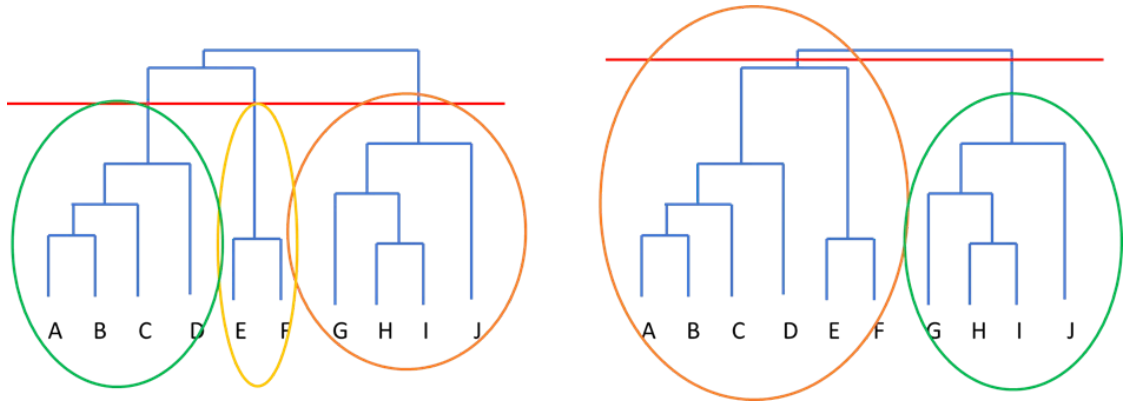


Figure 3 shows an example for dendrogram. We can see the process for Hierarchical clustering is to start from the bottom and merge data points that are closest to each other. It

then continues to merge the new clusters with other data points or clusters until it forms a big cluster for all data points. The next step is to decide how many clusters to form for these data points.

Figure 4. Dendrogram with Possible Cluster Choices



Choosing how many clusters to accept is more of an art than a science. In Figure 4, we show two options for the same dendrogram. The dendrogram on the left is divided in 3 clusters and the dendrogram on the right is divided in 2 clusters. We first draw a red line across the dendrogram. On the left dendrogram, the red line cuts across 3 vertical lines. All the data points connected to this vertical line forms a cluster. 3 vertical lines form 3 clusters as shown in Figure 4. We do the same thing for the dendrogram on the right except this time we draw the red line higher. Because we draw the red line higher it only cuts across 2 vertical lines. 2 vertical lines form 2 clusters. All the data points connected to a vertical line belong to the same cluster.

The way we check if our cluster choice makes sense is to investigate the common features in each cluster and see if we can identify distinct difference among different clusters. This is a trial-and-error process. We repeat the exercise with different numbers of clusters and then select the number that most suitable for the purpose of the project goal.

4.2. K -Means Procedure

Another popular clustering analysis method is **K -Means Procedure**. K in K -Means means the number of clusters to choose. For this method, we need to pre-select how many clusters we would like to have. Usually, we would select a few options and run through K -Means for each option. Then then we would conduct common features investigation for the cluster result as we describe in Hierarchical clustering. For example, for a clustering project, one could choose 3 clusters, 4 clusters and 5 clusters to run K -Means. One would run 3-cluster K -Means procedure, generate 3 clusters, investigate the features in 3 clusters to see if there are distinct differences among clusters and take a note. Then one would move to a 4-cluster K -Means procedure and repeat the same exercise until all cluster options are

done. Then choose the number of clusters that provide the most insights for the project goal.

How do we run K -Means procedure? Assume we are running 4-cluster K -Means procedure. K -Means will initially randomly split data points into 4 clusters. It will then calculate the mean of each cluster. Say we have cluster A , cluster B , cluster C and cluster D . The means for them are $\bar{A}$, $\bar{B}$, $\bar{C}$ and $\bar{D}$. Now we pick a data point a_1 from cluster A and calculate this point's distance to $\bar{A}$, $\bar{B}$, $\bar{C}$ and $\bar{D}$. If the distance between a_1 and $\bar{A}$ is actually longer than the distance between a_1 and $\bar{C}$, then we will reassign a_1 to cluster C . Then we recalculate the means of the 4 clusters and do the distance check again. We repeat the process until no data point being reassigned to a different cluster. This is the idea of how K -Means procedure runs. In the Machine Learning class, we will dive into the details of the K -Means algorithm.

5. Shiny Application for Monte Carlo Simulation

In this module, we provide a Shiny application to run a Monte Carlo simulation for an investment account balance.

[Click here to access the application.](#) In this application, you select the number of simulations. The histogram of the simulation will be generated automatically.

6. CONCLUSION

In this lesson, we gave a high-level overview of the basics of Machine Learning. We first introduced the relevant definitions and discussed the differences between supervising learning and unsupervised learning. Then we introduced one modeling method for each category. For supervised learning, we talked about classification, looked at linear discriminant analysis and provided one application of how it can be used in finance. Then we introduced cluster analysis for unsupervised learning. We gave a brief overview of hierarchical clustering and K-means procedure. This lesson offers some idea of what is covered in the Machine Learning class. This lesson completes the Financial Econometrics class.